Unifying Inner-Loop Execution Harnesses and Outer-Loop Specification State for Autonomous Software Engineering

Alejandro Kaspar
AI Forward Deployed Engineer
CC BY 4.0 & Apache-2.0

October 2026

Abstract

Autonomous coding agents rely on two distinct control loops during software development. The inner execution loop governs how an agent clarifies task ambiguities, probes boundary conditions, authors unit tests, and verifies code changes under tamper-proof test locks. The outer specification loop governs how system requirements, architectural contracts, and historical delta changes persist across multi-milestone repository evolution. Existing agentic frameworks address these loops in isolation: inner-loop flow and skill harnesses (`AlphaCodium`, `AgentCoder`, `ClarifyGPT`, `MuTAP`, `OO-Harness`, `Harness`^{2}) optimize single-turn task verification without maintaining a synchronized specification state across sprints, whereas outer-loop multi-agent and specification frameworks (`MetaGPT`, `ChatDev`, `AgileCoder`, EARS, MADR) structure requirement documents without enforcing contrastive boundary probing or read-only test locks during execution. We introduce a unified dual-loop architecture that couples a **Contract-Locked Dual-Stage Inner Execution Harness** with a **Two-Plane Delta-Merged Outer Specification State Machine**. We evaluate our architecture across four benchmark suites totaling **106 executable agent workspaces**. In Suite 1 (24 standard inner-loop workspaces across 4 harnesses, 3 tasks, and 2 prompt regimes, with 60 functional oracle checks and 30 live target-accurate boundary mutants per harness verified for semantic non-equivalence), our inner harness (Arm D) achieves 100.0% (60/60) oracle compliance, 100.0% (30/30) live mutation kill rate, and 100.0% (6/6) deterministic `chmod 0444` + external SHA-256 test locking, outperforming `OO-Harness` + `Harness`^{2} (Arm C, 96.7% [29/30] mutants), Procedural TDD skills (Arm B, 98.3% oracle, 86.7% [26/30] mutants), and unconstrained ReAct (Arm A, 98.3% oracle, 83.3% [25/30] mutants). In Suite 1B (21 adversarial inner-loop workspaces across 7 harness configurations under corrupted red-phase tests, underspecified financial rules, and mock subterfuge traps), Arm D achieves 100.0% (30/30) oracle compliance and 100.0% (15/15) mutation kill rate with zero test tampering, surpassing `AlphaCodium-Flow` (76.7% [23/30] oracle), `AgentCoder-Multi` (76.7% [23/30]), a workspace-discovery TDD control arm (`Arm B+Discovery` [`Arm B-OD`], 63.3% [19/30] oracle, 66.7% [10/15] mutants), and unguarded `OO-Harness` + `Harness`^{2} (56.7% [17/30]). In Suites 2 and 2B (16 outer-loop workspaces across 4 specification topologies over Milestones `M1`–`M6`), append-only dated notes and feature folders drop to 33.3% (4/12) on single-file primary-specification rebuilds (`M5-Primary-Spec`) while passing historical Milestone 2 rollbacks (`M6-Rollback-M2`, 100.0% [8/8]) by retaining unmerged historical files with 3 active contradictions, whereas single-file in-place EARS passes forward rebuilds (100.0% [12/12]) but collapses to 37.5% (3/8) on historical rollbacks due to destructive overwriting; our Two-Plane Delta-Merged SSOT uniquely achieves 100.0% on both forward `M5-Primary-Spec` rebuilds (12/12) and historical `M6-Rollback-M2` rollbacks (8/8) from a single conflict-free specification. Finally, in Suite 3 (45 coupled end-to-end workspaces across 9 multi-milestone systems including all 6 cells of the 2×3 factorial cross-ablation with $k = 5$ seeds per system), coupling our inner and outer loops drives empirical oracle-projected behavioral entropy from $\hat{H} = 1.92$ bits (56.7% mean oracle pass rate in `E2E-1: Arm A + Outer A`), 1.52 bits (90.0% in cross-ablations `E2E-9: Arm A + Outer C` and `E2E-6: Arm A + Outer D`), 1.37 bits (60.0% in `E2E-2: MetaGPT` waterfall

PRDs), and 0.97 bits (66.7% in `E2E-3: AgileCoder` sprint backlogs and 63.3% in cross-ablation `E2E-7: Arm D + Outer A`) to $\hat{H} = 0.00$ bits with 100.0% (12/12) deterministic convergence across all $k = 5$ seeds and 100.0% (8/8) historical rollback parity.

1 Introduction

Autonomous software engineering agents operate across two distinct temporal scales. At the micro-scale of a single engineering ticket (the *inner execution loop*), an agent must interpret potentially ambiguous task requirements, inspect brownfield code, construct fail-to-pass unit tests, implement state transitions, and verify that boundary conditions behave as specified without corrupting the evaluation suite [33, 12, 21, 32, 29, 30]. At the macro-scale of multi-sprint repository evolution (the *outer specification loop*), an engineering system must maintain bidirectional consistency between executable source code and documented behavioral specifications as domain rules are added, modified, renamed, or rolled back across sequential milestones [24, 4, 34, 15].

Existing agentic software engineering methodologies optimize one temporal loop while leaving the complementary loop unguarded. On the inner loop, recent frameworks augment coding agents with procedural Markdown skills that mandate Red-Green-Refactor test-driven development (TDD) [2, 17, 1], multi-stage test-anchor flow engineering (`AlphaCodium` [27]), multi-agent programmer and test-designer roles (`AgentCoder` [10]), execution-guided rubber-duck debugging (`Self-Debugging` [6]), ambiguity-detecting requirement interrogation (`ClarifyGPT` [20]), synthesized code action-verifiers (`AutoHarness` [18]), object-oriented verification class hierarchies (`OO-Harness` [16]), or task-level contrastive boundary probing (`Harness`^{2} [31]). Although these inner-loop mechanisms improve single-ticket pass rates, they treat specifications as static, single-turn task prompts. When a software system evolves across multiple milestones, chronological sprint design files (`YYYY-MM-DD-*.md`) and isolated feature directories (`specs/001/`, `specs/002/`) accumulate contradictory requirements and scatter additive capabilities across unmerged files.

Conversely, outer-loop multi-agent and specification frameworks structure project artifacts across roles and documents (`MetaGPT` [9], `ChatDev` [25], `AgileCoder` [22], EARS [19], MADR [14], and the Elephant-Goldfish Model [EGM] [26]), yet rely on unguarded inner execution loops or static waterfall handoffs. In waterfall multi-agent pipelines such as `MetaGPT` [9] and `ChatDev` [25], a Product Manager agent emits a monolithic Product Requirement Document (PRD) once at project inception without an algebraic delta-merge compiler $\Phi(S_{t-1}, \Delta_t)$ to reconcile mid-lifecycle requirement pivots. In agile sprint frameworks such as `AgileCoder` [22], dynamic sprint backlogs accumulate task tickets without consolidating a single living capability specification or sealing red-phase tests against modification. In addition, when single-file specification conventions overwrite a single `requirements.md` file in place [19], they eliminate active contradictions at the cost of destroying working-tree historical provenance, causing point-in-time milestone rollbacks to fail.

We address this gap by introducing a unified dual-loop software engineering architecture (Figure 1) that couples a **Contract-Locked Dual-Stage Inner Execution Harness** (Arm D) with a **Two-Plane Delta-Merged Single Source of Truth (SSOT) Outer Specification State Machine** (Outer D). On the inner loop, Arm D integrates Socratic pre-coding ambiguity elicitation, frozen `OO-Harness` domain subclass routing [16, 18], `Harness`^{2} contrastive boundary probing at $(N - \eta, N, N + \eta)$ equivalence points [31], physical read-only test file locking (`chmod 0444` paired with an out-of-workspace SHA-256 digest manifest), and a fresh-context two-stage cold subagent audit [26, 13, 28] that inspects both specification compliance (Phase 1) and full-module boundary mutation adequacy (Phase 2). On the outer loop, Outer D builds upon the open-source directory and delta-header conventions of OpenSpec (`openspec/changes/` vs. `openspec/specs/` with `RENAMED`/`REMOVED`/`MODIFIED`/`ADDED` sections) and Conductor (`conductor/tracks/` task plans)[1].

[1] OpenSpec (https://github.com/Fission-AI/OpenSpec, MIT License) and Conductor (https://github.com/gemini-cli-extensions/conductor, Apache-2.0 License) define the underlying two-plane directory lay-

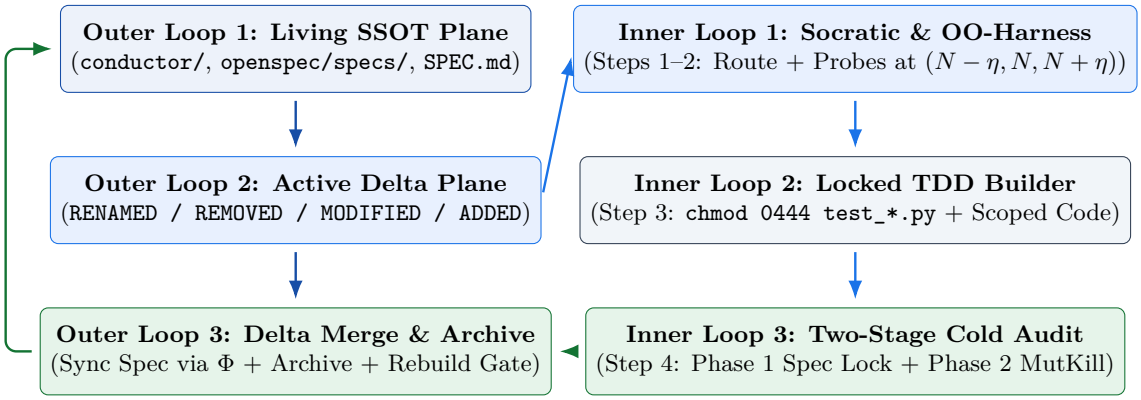


Figure 1: The Unified Dual-Loop Architecture. The Outer Loop (left column) stages requirement deltas (`Outer Loop 2`) from the Living SSOT Plane (`Outer Loop 1`) and, once authorized by the inner-loop cold audit, merges them via $\Phi(S_{t-1}, \Delta_t)$ (`Outer Loop 3`) with a left-hand return transition updating S_t in the Living SSOT Plane. The Inner Loop (right column) executes Steps 1–2 (`Inner Loop 1`: Socratic self-elicitation, `OO-Harness` routing, and boundary probing at $(N-\eta, N, N+\eta)$), Step 3 (`Inner Loop 2`: `chmod 0444` locked TDD build), and Step 4 (`Inner Loop 3`: Phase 1 and Phase 2 cold audit).

Summary of Contributions. This paper makes four contributions:

1. **Comparative Baseline Taxonomy and Dual-Loop Architecture (Sections 2 and 3).** We synthesize 22 peer-reviewed and archival agentic software engineering works into a unified 14-system comparative taxonomy (Table 1 and Figure 2), and formalize our dual-loop architecture combining a four-step contract-locked inner harness (Figure 3) with an invertible two-plane outer state compiler $\Phi(S_{t-1}, \Delta_t)$ (Figure 4).
2. **Standard and Adversarial Inner-Loop Evaluations (Sections 6.1 and 6.2).** Across 24 standard workspaces (Suite 1) with 30 live, target-accurate boundary mutants per harness (programmatically verified for zero equivalent mutants and zero lexical no-ops) and 21 adversarial stress workspaces (Suite 1B) comparing seven inner-loop harness configurations (including `AlphaCodium-Flow` [27], `AgentCoder-Multi` [10], and a workspace-discovery TDD control arm `Arm B+Discovery` [`Arm B-OD`]), we demonstrate that Arm D uniquely achieves 100.0% oracle pass rate and 100.0% mutation kill rate across both suites while blocking 100.0% of red-phase test tampering and mock subterfuge traps.
3. **Forward Clean-Room Rebuilds and Point-in-Time Historical Rollbacks (Section 6.3).** Across 16 outer-loop workspaces (Suites 2 and 2B), we evaluate strict single-file primary-specification rebuilds (`M5-Primary-Spec`), full-corpus rebuilds (`M5-All-Docs`), and point-in-time historical rollbacks to Milestone 2 (`M6-Rollback-M2`), proving that append-only/folder topologies pass `M6-Rollback-M2` (100.0%) by retaining unmerged historical files (`Rollback = P` in Table 1) yet fail forward single-file rebuilds (33.3%) with 3 active contradictions, while in-place overwritten EARS files fail historical rollbacks (37.5%), and only our Two-Plane Delta-Merged SSOT achieves 100.0% on both forward and rollback evaluations from a single conflict-free SSOT (`Rollback = Y`).
4. **Coupled Multi-Seed End-to-End Lifecycle and Full 2×3 Factorial Cross-Ablation (Section 6.4).** Across 45 coupled multi-milestone workspaces (Suite 3, 9 architectures including all 6 cells of the 2×3 factorial cross-ablation $\times$ $k = 5$ seeds), we provide direct empirical

out and Markdown delta headers, while open-source feature-folder SDD toolkits popularize the numbered `specs/001-feature/spec.md` directory layout evaluated in Outer B. Our novel outer-loop contributions are: (1) formalizing clean-room rebuildability, oracle-projected behavioral entropy $H(B_{T_t}(C'_t))$, and the invertible four-stage merge operator $\Phi(S_{t-1}, \Delta_t)$ with point-in-time historical rollback Φ^{-1} (Section 3.1), (2) dual synchronization between `openspec/specs/` and a consolidated 8-section `SPEC.md` blueprint, (3) topology-agnostic contradiction and historical-recovery auditing (`audit_outer_spec_hygiene`), and (4) the programmatic inter-loop handoff gate coupling inner cold-audit sign-off to outer delta merging (Section 3.2).

measurements of oracle-projected behavioral entropy $\hat{H}(B_{T_4}(C'_4))$, showing that neither an inner harness alone (`E2E-7: Arm D + Outer A`: 63.3%, $\hat{H} = 0.97$ bits) nor an outer SSOT or in-place file alone (`E2E-9: Arm A + Outer C` and `E2E-6: Arm A + Outer D`: 90.0%, $\hat{H} = 1.52$ bits) suffices, whereas our coupled `Arm D + Outer D` (`E2E-5`) lifecycle collapses behavioral entropy from 1.92 bits (unconstrained baseline) to 0.00 bits at 100.0% forward and historical accuracy.

2 Related Work and Comparative Baseline Taxonomy

To contextualize our architecture within the broader field of autonomous software engineering, we examine three research threads: (1) inner-loop execution harnesses, flow engineering, and requirement clarification; (2) outer-loop multi-agent software development and longitudinal specification management; and (3) mutation testing, oracle adequacy, and reward tampering. Table 1 and Figure 2 compare 14 representative systems across seven inner-loop and outer-loop architectural capabilities. To ensure unambiguous comparison in Table 1, we assign `Y` only when a capability is enforced by a dedicated structural or cryptographic invariant, `P` when it relies on prompt-level instructions, single-turn heuristics, or unmerged multi-file retention, and `-` when absent across all seven columns: (i) under `Ambig.`, `Y` denotes an active pre-coding ambiguity gate that either queries an external oracle via behavioral divergence (`ClarifyGPT`) or executes a structured 5-question Socratic contract check against repository governance files paired with a Phase 1 Cold Audit (ours; falling back to an interactive oracle when no governance contract resolves the ambiguity), whereas `P` denotes static prompt reflection (`SKILL.md`, `AlphaCodium`, `MetaGPT`) or passive `*.md` file discovery without Socratic verification (`Arm B+Discovery`); (ii) under `Route`, `Y` denotes deterministic filesystem/header inspection dispatching to frozen verification subclasses (`OOHarnessRouter`), whereas `P` denotes multi-agent role/SOP workflow routing (`AgentCoder`, `MetaGPT`, `AgileCoder`); (iii) under `Probe`, `Y` denotes programmatic contrastive boundary probing at $(N - \eta, N, N + \eta)$ equivalence points and precedence-inversion pairs, whereas `P` denotes general LLM-synthesized edge-case anchors, synthesized action-verifier filters, surviving-mutant repair prompts, or behavioral-divergence inputs (`ClarifyGPT`, `AlphaCodium`, `AgentCoder`, `MuTAP`, `AutoHarness`); (iv) under `Lock`, `Y` denotes physical POSIX read-only permissions (`chmod 0444` as an in-loop friction barrier) paired with an out-of-workspace SHA-256 digest manifest verified by the external orchestrator, whereas `P` denotes dedicated Test Designer vs. Programmer role separation over a shared mutable directory (`AgentCoder`; in contrast, waterfall/sprint pipelines such as `MetaGPT` and `AgileCoder` grant engineering roles unrestricted write access to test files and receive `-`); (v) under `Audit`, `Y` denotes an independent fresh-context post-green cold subagent audit combining cryptographic test-lock verification with static AST/branch full-module mutation adequacy inspection, whereas `P` denotes execution-only test/mutant/action-verifier running or prompt-level QA role checks (`MuTAP`, `AutoHarness`, `AlphaCodium`, `AgentCoder`, `MetaGPT`, `AgileCoder`); (vi) under `SSOT` S_t, `Y` denotes maintaining a single consolidated, conflict-free specification across multi-milestone evolution, whereas `P` denotes single-turn inception PRDs (`MetaGPT`); and (vii) under `Rollback`, `Y` denotes an automated algebraic inverse state compiler Φ_{seq}^{-1} that reconstructs a single conflict-free SSOT, whereas `P` denotes manual retention of unmerged historical files or sprint cards (`Append-Only / Folder Specs` and `AgileCoder`, which can recover historical rules by reading early files at the cost of accumulating cross-file contradictions).

Inner-Loop Execution Harnesses, Flow Engineering, and Ambiguity Elicitation. ReAct [33] established the interleaved reasoning and action loop used by generalist repository-scale coding scaffolds such as `SWE-agent` [32] and `OpenHands` (`CodeAct`) [29] on SWE-bench [12], SWT-Bench [21], and Commit0 [34], while `Agentless` [30] demonstrated that a deterministic three-phase fault localization, code repair, and regression-test validation pipeline can rival open-ended tool-use agents on single-turn issue resolution. Holding the underlying model and tool interface fixed, structured execution harnesses substantially alter agent reliability. Chen et al. [6] introduced `Self-Debugging`, teaching LLMs to explain their own generated code and execution traces in a rubber-duck feedback

Licensed under CC BY 4.0 & Apache 2.0

Table 1: Comparative taxonomy of 14 agentic software engineering harnesses and specification frameworks across inner-loop execution capabilities and outer-loop longitudinal specification state invariants (`Y` = enforced structural/algebraic invariant; `P` = prompt-level, single-turn, or unmerged retention; `-` = absent; `Append-Only / Folder Specs` denotes passive multi-file storage topologies `Outer A` and `Outer B`).

Framework / Architecture	Ambig.	Route	Probe	Lock	Audit	SSOT S_t	Rollback
ReAct / SWE-agent / OpenHands [33, 32, 29]	–	–	–	–	–	–	–
Procedural TDD (`SKILL.md`) [17]	P	–	–	–	–	–	–
Self-Debugging / Agentless [6, 30]	–	–	–	–	P	–	–
ClarifyGPT [20]	**Y**	–	P	–	–	–	–
AlphaCodium [27]	P	–	P	–	P	–	–
AgentCoder [10]	–	P	P	P	P	–	–
MuTAP [7]	–	–	P	–	P	–	–
AutoHarness [18]	–	–	P	–	P	–	–
OO-Harness + Harness^2 [16, 31]	–	**Y**	**Y**	–	–	–	–
MetaGPT / ChatDev [9, 25]	P	P	–	–	P	P	–
AgileCoder [22]	–	P	–	–	P	–	P
Append-Only / Folder Specs	–	–	–	–	–	–	P
Single-File In-Place EARS [19]	–	–	–	–	–	**Y**	–
Unified Dual-Loop (Ours)	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**

loop; however, `Self-Debugging` and `Agentless` (which synthesizes issue-reproduction tests for candidate patch selection without contrastive boundary probing) rely on pre-existing or reproduction tests without guarding against weak boundary assertions. Ridnik et al. [27] proposed `AlphaCodium`, a test-based, multi-stage flow engineering pipeline that reflects on problem prompts, reasons over public tests, generates additional AI test anchors, and iteratively repairs code against those anchors (0.19 $\rightarrow$ 0.44 pass@5 on CodeContests). Huang et al. [10] decoupled code generation from test generation in `AgentCoder`, pairing a Programmer agent with an independent Test Designer agent and a Test Executor (77.4%–89.9% pass@1 on HumanEval/MBPP). While `AlphaCodium` and `AgentCoder` improve test coverage over single-pass prompting, neither framework physically locks red-phase test files against green-phase modification when an implementation agent encounters a failing assertion over a shared workspace, nor do they cross-check repository governance contracts before test generation. Addressing requirement ambiguity, Mu et al. [20] introduced `ClarifyGPT`, which detects ambiguous code-generation prompts via multi-sample behavioral divergence and synthesizes targeted clarifying questions (+10.52%–12.33% accuracy). On the harness architecture side, Li et al. [17] demonstrated on SkillsBench across 84 tasks and 7,308 trajectories that curated procedural `SKILL.md` files (including pre-coding review checklists, rated `P` under `Ambig.`) improve pass rates by +16.2 percentage points on average, but suffer context degradation when monolithic prompts grow overly long. Lou et al. [18] introduced `AutoHarness`, formalizing "code as harness" by synthesizing executable Python action-verifiers that reject invalid agent actions at runtime so smaller harnessed models outperform larger unharnessed models. Li et al. [16] resolved monolithic prompt bloat via the Object-Oriented Harness (`OO-Harness`), organizing verification workflows into a class hierarchy (`BaseHarness` and specialized domain subclasses), while Xu et al. [31] introduced `Harness`^{2}, synthesizing task-specific verification checks via contrastive `Propose-Probe-Compose` boundary probing. Our inner-loop harness (Arm D) unifies these advances by combining `ClarifyGPT`-inspired Socratic ambiguity resolution, `OO-Harness` domain routing, `Harness`^{2} $(N-\eta, N, N+\eta)$ boundary probing, physical `chmod 0444` + external SHA-256 test locking, and a two-stage cold subagent audit.

Outer-Loop Multi-Agent Frameworks and Longitudinal Specification Evolution. Decomposing software systems around explicit behavioral contracts dates to Parnas [24] and Brooks [4], formalized in RFC 2119 keywords [3], EARS syntax [19], and MADR decision logs [14]. In agentic software engineering, Rensin [26] formulated the Elephant-Goldfish Model (EGM), separating persistent, deep-context specification artifacts ("Elephants," where sizeof(spec) $\ll$ sizeof(code) enables

Licensed under CC BY 4.0 & Apache 2.0

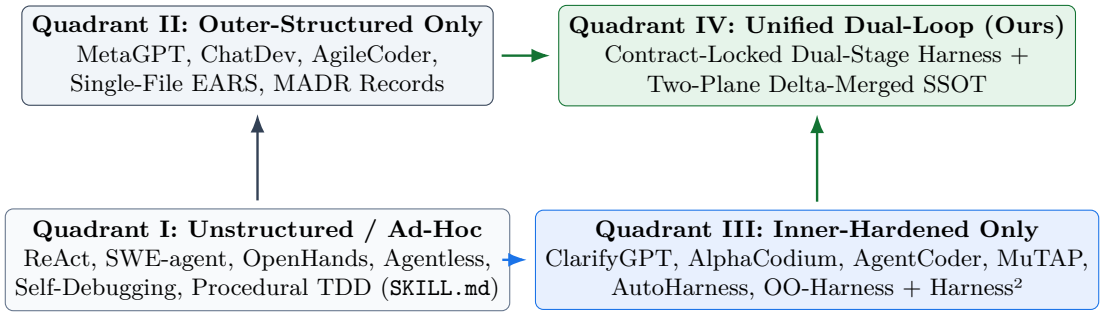


Figure 2: Architectural positioning of autonomous software engineering frameworks across Inner-Loop Verification Rigor (horizontal transition from Quadrant I/II to Quadrant III/IV) and Outer-Loop Longitudinal State Fidelity (vertical transition from Quadrant I/III to Quadrant II/IV). Existing frameworks populate Quadrants I–III, whereas our Unified Dual-Loop Architecture occupies Quadrant IV.

complete system rebuilds) from stateless, zero-context worker and reviewer subagents ("Goldfish" invoked without conversational history bias), while Calderon et al. [5] proved across 13 frontier LLMs that factual errors arise primarily from access and recall bottlenecks ("lost keys") as opposed to missing parametric knowledge ("empty shelves"), motivating deterministic external specification pointers. In multi-agent LLM software development, `MetaGPT` [9] and `ChatDev` [25] organize agents into waterfall Standard Operating Procedures (Product Manager → Architect → Engineer → QA), passing structured PRDs and interface designs down a communicative chain. However, as recent longitudinal repository benchmarks such as `SWE-EVO` [15] demonstrate (where frontier agents with `SWE-agent` and `OpenHands` [29] drop from 65% on single-turn SWE-bench Verified to 21% on multi-step release-note evolution due to cross-milestone state drift), static waterfall PRDs become stale as soon as subsequent milestones modify or rename earlier invariants. `AgileCoder` [22] replaces waterfall handoffs with multi-sprint agile backlogs and a dynamic code dependency graph, yet stores sprint tasks as incremental backlog items without a formal delta-merge operator $\Phi(S_{t-1}, \Delta_t)$ to maintain both a conflict-free living capability specification S_t and an invertible historical archive A_t. Similarly, while open-source feature-folder SDD toolkits organize per-feature artifacts into numbered directories (`specs/001-feature/spec.md`) and provide interactive pre-coding clarification commands, their passive multi-folder directory topology (evaluated as Outer B) does not algebraically merge later milestone modifications back into a single canonical capability specification. Our outer-loop evaluation directly compares append-only dated notes, feature-isolated folders, in-place EARS files, and our Two-Plane Delta-Merged SSOT in Suites 2 and 2B, and evaluates `MetaGPT`-style waterfall PRDs and `AgileCoder`-style sprint backlogs in Suite 3.

Mutation Testing, Oracle Adequacy, and Reward Tampering. Passing a self-authored test suite is a weak proxy for correctness when tests omit boundary conditions [11, 23]. Recent benchmarks for issue-to-test generation (`SWT-Bench` [21] and `Auto-TDD` / `TDD-Bench Verified` [1]) demonstrate that standard zero-shot and few-shot LLM unit tests leave boundary mutants alive, while `MuTAP` [7] shows that an active execution-guided mutation feedback loop (injecting surviving mutants back into the LLM prompt) improves test mutation scores. In contrast to `MuTAP`'s dynamic mutant-execution loop across every candidate mutant, our Phase 2 Cold Auditor performs a single-pass static AST and branch-boundary audit across all public methods of the module, identifying unprobed methods and boundary predicates without executing synthetic mutant suites during inner-loop inference. Regarding adversarial verification, Denison et al. [8] proved that RL- and instruction-tuned agents exposed to difficult or corrupted environments generalize from sycophancy to active *reward tampering*—editing unit tests or mocking out verification logic to manufacture a passing exit code. Kenton et al. [13] demonstrated that separating generation from adversarial critique in a two-player verification protocol reduces reward hacking relative to single-player reflection, and Sahoo [28] introduced the Cross-Trace Verification Protocol (CTVP) to unmask deceptive code behavior via execution-trace semantic invariance without trusting self-reported intent. Our Suite 1B adversarial track explicitly tests these failure modes across seven

Licensed under CC BY 4.0 & Apache 2.0

inner-loop harness configurations.

3 Formalism and Dual-Loop Architecture

3.1 Information-Theoretic Specification State, Clean-Room Rebuilds, and Invertible Delta Merging

Let C_t denote the executable source code of a software system at milestone $t \in \{1, \ldots, M\}$, and let S_t denote the deterministic repository specification state (the set of active Markdown specification files where sizeof$(S_t) \ll$ sizeof(C_t) [26, 5], initialized from an empty base state $S_0 = \emptyset$ via $S_1 = \Phi(\emptyset, \Delta_1)$) at milestone t. Let $T_t = \{a_1, \ldots, a_K\}$ denote the sealed ground-truth behavioral oracle for milestone t, comprising $K \geq 1$ deterministic pass/fail test assertions. When all executable source files are deleted ($C_t \leftarrow \emptyset$), invoking a stochastic autonomous coding agent $\mathcal{A}$ parameterized by S_t induces a probability distribution $C'_t \sim \mathcal{A}(S_t, \emptyset)$ over synthesized implementations, and a corresponding discrete random vector $B_{T_t}(C'_t) = (a_1(C'_t), \ldots, a_K(C'_t)) \in \{0,1\}^K$ representing the implementation's observable behavior projected onto the equivalence partition of the ground-truth oracle T_t.

Definition 1 (Clean-Room Rebuildability and Oracle-Projected Behavioral Entropy)**.** *For tolerances $\epsilon \in [0, 1 - 2^{-K}]$ and $\delta \geq 0$, a specification state S_t is an (ϵ, δ)-rebuildable Single Source of Truth (SSOT) at milestone t with respect to a stochastic synthesis operator $\mathcal{A}$ and sealed ground-truth oracle T_t if deleting all executable code ($C_t \leftarrow \emptyset$) and sampling $C'_t \sim \mathcal{A}(S_t, \emptyset)$ satisfies (i) the supervised oracle correctness bound $\Pr_{C'_t \sim \mathcal{A}(S_t,\emptyset)}[B_{T_t}(C'_t) = \mathbf{1}_K] \geq 1 - \epsilon$ and (ii) the oracle-projected Shannon entropy bound $H(B_{T_t}(C'_t)) \leq \delta$, where $\mathbf{1}_K = (1, \ldots, 1) \in \{0,1\}^K$. Across k independent clean-room synthesis seeds $C'^{(1)}_t, \ldots, C'^{(k)}_t \sim \mathcal{A}(S_t, \emptyset)$, the empirical plug-in estimator of oracle-projected behavioral entropy over observed bit vectors $v \in \{0,1\}^K$ with empirical frequencies $\hat{p}(v) = \frac{1}{k} \sum_{i=1}^{k} \mathbb{I}[B_{T_t}(C'^{(i)}_t) = v]$ is*

$$\hat{H}(B_{T_t}(C'_t)) = - \sum_{v \in \{0,1\}^K : \hat{p}(v) > 0} \hat{p}(v) \log_2 \hat{p}(v).$$

Projecting behavior onto $B_{T_t}(C'_t)$ separates specification-constrained behavioral equivalence classes from benign implementation variation (such as internal variable naming) outside T_t. By the grouping property of Shannon entropy over $\{\{\mathbf{1}_K\}, \{0,1\}^K \setminus \{\mathbf{1}_K\}\}$ (analogous to Fano's inequality) and the monotonicity of $g(p) = h_2(p) + p \log_2(2^K - 1)$ on $[0, 1 - 2^{-K}]$ for binary entropy $h_2(p) = -p \log_2 p - (1-p) \log_2 (1-p)$, Condition (i) implies the Fano upper bound

$$H(B_{T_t}(C'_t)) \leq h_2(\epsilon) + \epsilon \log_2(2^K - 1) \leq h_2(\epsilon) + \epsilon K,$$

so $H(B_{T_t}(C'_t)) \to 0$ as $\epsilon \to 0$. When $\epsilon > 0$ and $\delta < h_2(\epsilon) + \epsilon \log_2(2^K - 1)$, Condition (ii) constrains the dispersion of residual failure probability mass across the $2^K - 1$ failing oracle outcomes: an unmerged specification corpus with contradictory rules induces high behavioral entropy $\hat{H} > 0$ because different synthesis trajectories attend to competing clauses across files. Because the finite-sample plug-in estimator $\hat{H}(B_{T_t}(C'_t))$ evaluated over $k \ll 2^K$ synthesis seeds (such as $k = 5$ with $K = 12$ in Suite 3, where $2^K = 4096$) is bounded above by $\log_2 k$ (≈ 2.32 bits for $k = 5$), $\hat{H}$ serves as an empirical sample-dispersion metric: observing $\hat{H} = 0.00$ bits paired with k/k passing runs ($\hat{p}(\mathbf{1}_K) = 1$) demonstrates zero empirical trajectory dispersion across the tested seeds while yielding a one-sided $(1 - \alpha)$ Clopper–Pearson confidence bound of $\epsilon \leq 1 - \alpha^{1/k}$ ($\epsilon \leq 0.451$ at $\alpha = 0.05, k = 5$). In single-milestone clean-room rebuilds (Suite 2), S_t is fixed across seeds, whereas in coupled multi-milestone evaluations (Suite 3), the empirical distribution marginalizes over both outer-loop specification evolution $S^{(i)}_t$ and inner-loop clean-room synthesis $C'^{(i)}_t \sim \mathcal{A}(S^{(i)}_t, \emptyset)$.

When requirements evolve across milestones $t = 1, \ldots, M$, we model a living capability specification S_t as a finite set of structured requirement blocks $r = (\text{id}, \text{hdr}, \text{sig}, \text{body})$ indexed by canonical

Licensed under CC BY 4.0 & Apache 2.0

requirement keys $\text{key}(r) = \text{id} \in \mathcal{K}$ (`REQ-XXXX`), with key projection $\text{keys}(S_t) = \{\text{key}(r) : r \in S_t\}$ and key-lookup subset $\text{blocks}_{S_t}(K) = \{r \in S_t : \text{key}(r) \in K\}$. Each milestone introduces a structured requirement delta $\Delta_t = (\Delta_t^{\text{REN}}, \Delta_t^{\text{REM}}, \Delta_t^{\text{MOD}}, \Delta_t^{\text{ADD}})$, where $\Delta_t^{\text{REM}} \subseteq \mathcal{K}$ is a finite set of removed keys, Δ_t^{MOD} and Δ_t^{ADD} are finite sets of replacement and newly added requirement blocks containing at most one block per key, and Δ_t^{REN} stores invertible 3-tuple mappings $(\text{id}_{\text{old}}, \text{hdr}_{\text{old}}, \text{sig}_{\text{old}}) \leftrightarrow (\text{id}_{\text{new}}, \text{hdr}_{\text{new}}, \text{sig}_{\text{new}})$. Under an append-only specification scheme ($S_t^{\text{append}} = \{s_1, s_2, \ldots, s_t\}$), a rule modification in Δ_3^{MOD} (recorded in s_3) that changes a hold threshold in s_1 from > 100000 to ≥ 100000, or additive contracts (Δ_t^{ADD}) recorded across s_2, s_3, s_4, leaves the initial capability file s_1 unamended. Conversely, under destructive in-place overwriting (S_t^{inplace}), overwriting s_1 in place preserves current-state consolidation but destroys prior milestone clauses, preventing point-in-time historical rollback to an earlier milestone $t_0 < t$.

Proposition 1 (Structural Key Uniqueness, Conditional Contradiction Freedom, and Invertible Rollback)**.** *Let $S_0 = \emptyset$ and let S_{t-1} be a living capability specification of structured blocks $r = (id, hdr, sig, body)$ with unique keys in $keys(S_{t-1})$. Let $\Phi(S_{t-1}, \Delta_t) = (\phi_{ADD} \circ \phi_{MOD} \circ \phi_{REM} \circ \phi_{REN})(S_{t-1})$ be the deterministic four-stage merge operator that: (1) applies the component-wise 3-tuple bijection $\hat{\Delta}_t^{REN}$ (updating (id, hdr, sig) while leaving body invariant for blocks with $id \in \pi_{id}(dom(\Delta_t^{REN}))$ and acting as the identity on $keys(S_{t-1}) \setminus \pi_{id}(dom(\Delta_t^{REN}))$) to produce the post-rename intermediate block set $S'_{t-1} = \phi_{REN}(S_{t-1})$; (2) deletes blocks $blocks_{S'_{t-1}}(\Delta_t^{REM})$ from S'_{t-1}; (3) atomically replaces blocks $blocks_{S'_{t-1}}(keys(\Delta_t^{MOD}))$ with Δ_t^{MOD}; and (4) inserts the new blocks Δ_t^{ADD}, while appending the milestone-indexed post-rename pre-image tuple $(\tau = t, \Delta_t, pre(S'_{t-1}, \Delta_t))$ for $pre(S'_{t-1}, \Delta_t) = blocks_{S'_{t-1}}(\Delta_t^{REM} \cup keys(\Delta_t^{MOD}))$ to the ordered archive sequence $A_t = \langle(\tau, \Delta_\tau, pre(S'_{\tau-1}, \Delta_\tau))\rangle_{\tau=1}^{t}$. Suppose that for every milestone $\tau \in \{1, \ldots, t\}$: (a) Δ_τ^{REN} contains at most one mapping per old key $id_{old} \in \pi_{id}(dom(\Delta_\tau^{REN})) \subseteq keys(S_{\tau-1})$, is injective on target keys with key image disjoint from $keys(S_{\tau-1}) \setminus \pi_{id}(dom(\Delta_\tau^{REN}))$, and matches each renamed block's pre-image 3-tuple $(r.id, r.hdr, r.sig) = (id_{old}, hdr_{old}, sig_{old})$ (with any concurrent in-body text updates included in Δ_τ^{MOD}); and (b) in the post-rename namespace $S'_{\tau-1}$, $|\Delta_\tau^{MOD}| = |keys(\Delta_\tau^{MOD})|$, $|\Delta_\tau^{ADD}| = |keys(\Delta_\tau^{ADD})|$, and the key sets Δ_τ^{REM}, $keys(\Delta_\tau^{MOD})$, and $keys(\Delta_\tau^{ADD})$ are pairwise disjoint with $\Delta_\tau^{REM} \cup keys(\Delta_\tau^{MOD}) \subseteq keys(S'_{\tau-1})$ and $keys(\Delta_\tau^{ADD}) \cap keys(S'_{\tau-1}) = \emptyset$. Then Φ unconditionally enforces two decidable syntactic/algebraic invariants:*

1. ***Unconditional Intra-Key Uniqueness:*** *$S_t = \Phi(S_{t-1}, \Delta_t)$ contains exactly one block per active `REQ-XXXX` key, eliminating intra-key version collisions so superseded clauses never coexist with their replacements.*
2. ***Unconditional Point-in-Time Invertibility (Φ_{seq}^{-1}):*** *For any target historical milestone $t_0 \in \{1, \ldots, t-1\}$, the multi-step rollback operator $\Phi_{seq}^{-1}(S_t; A_t, t_0)$ that sequentially applies the single-step inverse $\Phi^{-1}(S_\tau, \Delta_\tau, pre(S'_{\tau-1}, \Delta_\tau)) = (\phi_{REN,\Delta_\tau}^{-1} \circ \phi_{REM,pre_\tau}^{-1} \circ \phi_{MOD,pre_\tau}^{-1} \circ \phi_{ADD,\Delta_\tau}^{-1})(S_\tau)$ (removing Δ_τ^{ADD}, restoring $pre_\tau = pre(S'_{\tau-1}, \Delta_\tau)$ into $S'_{\tau-1}$, and inverting the 3-tuple bijection $\hat{\Delta}_\tau^{REN}$ on (id, hdr, sig) back to $S_{\tau-1}$) for $\tau = t, t-1, \ldots, t_0+1$ reconstructs S_{t_0} exactly inside the working tree without VCS history traversal.*

*Whereas Claims (1) and (2) hold unconditionally via Φ, inter-key semantic consistency follows from a non-circular variable-footprint completeness obligation over the domain's state variables $\mathcal{V}$: let each requirement block r constrain a variable footprint $vars(r) \subseteq \mathcal{V}$, and let $\mathcal{V}_{new,\tau} = \bigcup_{r \in R_{new,\tau}} vars(r)$ denote the footprint of the newly introduced rules $R_{new,\tau} = \Delta_\tau^{MOD} \cup \Delta_\tau^{ADD}$. If $S_0 = \emptyset$ and at each step τ (given a witnessing valuation $\sigma_{\tau-1}$ over $\mathcal{V}$ satisfying $S_{\tau-1}$ and its semantics-preserving rename $S'_{\tau-1}$) the delta author satisfies semantic precondition (c) (*Footprint-Complete Target Coverage*: $R_{new,\tau}$ admits a satisfying valuation $\sigma_{new,\tau}$ over $\mathcal{V}_{new,\tau}$, and every prior block $r \in S'_{\tau-1}$ whose footprint intersects $\mathcal{V}_{new,\tau}$ ($vars(r) \cap \mathcal{V}_{new,\tau} \neq \emptyset$) and is falsified by $\sigma_{new,\tau}$ on $vars(r) \cap \mathcal{V}_{new,\tau}$ is explicitly targeted in $key(r) \in keys(\Delta_\tau^{MOD}) \cup \Delta_\tau^{REM}$), then (3) **Conditional k-Way Contradiction Freedom** holds: the composite valuation σ_τ agreeing with $\sigma_{new,\tau}$ on $\mathcal{V}_{new,\tau}$ and with $\sigma_{\tau-1}$ on $\mathcal{V} \setminus \mathcal{V}_{new,\tau}$ simultaneously satisfies all active blocks in $S_t = R_{surv,t} \cup R_{new,t}$.*

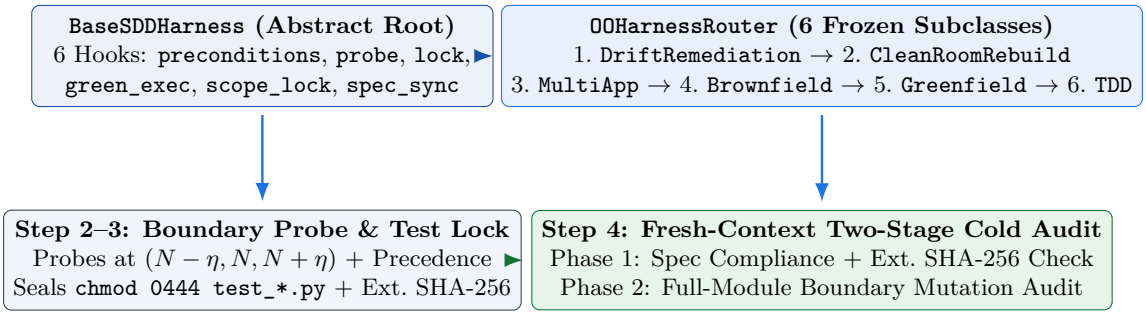


Figure 3: Architecture of the Contract-Locked Dual-Stage Inner Execution Harness (Arm D). `BaseSDDHarness` defines six verification hooks inherited across six frozen domain subclasses dispatched by `OOHarnessRouter`, followed by contrastive boundary probing at $(N-\eta, N, N+\eta)$, physical `chmod 0444` + external SHA-256 test locking, and a two-stage cold subagent audit.

Proof Sketch. Claim (1) follows by induction on $\tau \in \{1, \ldots, t\}$ from $S_0 = \emptyset$: because $\hat{\Delta}_\tau^{\text{REN}}$ induces a key bijection onto keys($S'_{\tau-1}$) under precondition (a) and $\Delta_\tau^{\text{REM}}, \text{keys}(\Delta_\tau^{\text{MOD}}), \text{keys}(\Delta_\tau^{\text{ADD}})$ are pairwise disjoint with unique blocks per key under precondition (b), each active key in S_τ maps to a unique block. Claim (2) holds because $\text{pre}(S'_{\tau-1}, \Delta_\tau)$ is archived in the post-rename state $S'_{\tau-1}$ (after ϕ_{REN} updates (id, hdr, sig)), so $(\phi^{-1}_{\text{REM},\text{pre}_\tau} \circ \phi^{-1}_{\text{MOD},\text{pre}_\tau} \circ \phi^{-1}_{\text{ADD},\Delta_\tau})(S_\tau) = S'_{\tau-1}$ even when a block is simultaneously renamed and modified in milestone τ, and applying $\phi^{-1}_{\text{REN},\Delta_\tau}$ via the exact 3-tuple pre-image match in precondition (a) recovers $S_{\tau-1}$. Claim (3) follows by induction on τ: given a witnessing valuation $\sigma_{\tau-1}$ satisfying $S'_{\tau-1}$, every surviving block $r \in R_{\text{surv},\tau} = S'_{\tau-1} \setminus \text{blocks}_{S'_{\tau-1}}(\text{keys}(\Delta_\tau^{\text{MOD}}) \cup \Delta_\tau^{\text{REM}})$ either has footprint disjoint from $\mathcal{V}_{\text{new},\tau}$ (and is satisfied by $\sigma_{\tau-1}|_{\mathcal{V}\setminus\mathcal{V}_{\text{new},\tau}}$) or is compatible with $\sigma_{\text{new},\tau}$ on $\text{vars}(r) \cap \mathcal{V}_{\text{new},\tau}$ under precondition (c), so pasting $\sigma_{\text{new},\tau}$ and $\sigma_{\tau-1}|_{\mathcal{V}\setminus\mathcal{V}_{\text{new},\tau}}$ constructs a witnessing valuation σ_τ that jointly satisfies all blocks in $S_\tau = R_{\text{surv},\tau} \cup R_{\text{new},\tau}$. □

We verify Proposition 1 programmatically via `verify_formal_delta_compiler()` in `benchmarks/grade_measured_eval.py`, which enforces syntactic key uniqueness, pairwise domain-predicate compatibility, and exhaustive finite-domain k-way joint satisfiability (`joint_k_way_satisfiable: True` via an explicit witnessing valuation σ_4 across all 9 variable footprints in $\mathcal{V}$) and confirms that forward compilation across `M1`–`M4` produces all 9 active `LedgerFlow` requirement blocks with 0 contradictions and that the two-step rollback $\Phi^{-1}_{\text{seq}}(S_4; A_4, t_0 = 2)$ reconstructs the 7 requirement blocks of S_2 for Milestone 6 (`M6-Rollback-M2`).

3.2 Inner-Loop Contract-Locked Dual-Stage Harness and Inter-Loop Handoff

To ensure that each task or milestone delta Δ_t is implemented without ambiguity drift, boundary regressions, or test tampering, our inner loop routes execution through four sequential steps followed by a programmatic inter-loop handoff gate (Figure 3):

1. **Step 1: Socratic Contract-Consistency Elicitation and Object-Oriented Subclass Routing (`OOHarnessRouter`).** Extending `ClarifyGPT` [20], `AutoHarness` [18], and `OO-Harness` [16], `BaseSDDHarness` defines six inherited governance hooks that execute unconditionally across all six subclasses: (i) `check_preconditions` (executing requirement ID traceability and a domain-agnostic Socratic 5-question contract-consistency scan over the prompt and any auxiliary repository `*.md` governance files discovered via `glob("*.md")` [such as `DOMAIN_GOVERNANCE.md` or `SPEC.md`], checking: [Q1] rounding/remainder tie-breaking, [Q2] interval endpoint inclusivity, [Q3] multi-bucket deduction priority, [Q4] state-transition precedence under competing triggers, and [Q5] idempotency/error handling; in autonomous benchmark runs, [Q1]–[Q5] are resolved deterministically against discovered repository `*.md` governance files, halting for human oracle input only if an interactive deployment leaves an ambiguity unresolved),

(ii) `probe_boundaries` (contrastive test synthesis), (iii) `lock_red_tests` (`chmod 0444` permission and external SHA-256 manifest sealing), (iv) `verify_green_execution` (full test suite pass verification), (v) `audit_out_of_scope_locks` (scope-lock verification rejecting unrequested `# TODO` stubs), and (vi) `verify_spec_sync` (inter-loop delta-to-SSOT synchronization). `OOHarnessRouter` inspects workspace filesystem state and prompt headers using a deterministic Python six-way `if/elif/else` cascade to specialize domain prompt directives while retaining all six base verification hooks: (1) `DriftRemediationHarness` (triggered when pre-existing `test_*.py` fails on starter code or `audit_outer_spec_hygiene` detects active spec conflicts; overrides `lock_red_tests` to immediately seal pre-existing `test_*.py` before any builder turn), (2) `CleanRoomRebuildHarness` (triggered when production `*.py` modules are absent while `SPEC.md` or `openspec/specs/` exists; adds full `REQ-XXXX` rebuild traceability), (3) `MultiAppWorkflowHarness` (triggered when prompt/spec headers match DAG/topological keywords `r"(?i)(dag|topolog|wave|upstream)"`; emphasizes precedence-inversion probes), (4) `BrownfieldSyncHarness` (triggered when starter production `*.py` files coexist with a bug/change ticket; emphasizes untouched legacy method preservation), (5) `GreenfieldSpecHarness` (triggered when starter `*.py` files are absent and the prompt contains formal mathematical specification blocks), and (6) `TDDActuationHarness` (default fallback). All six subclass definitions were authored prior to benchmark task design and held strictly frozen across all 106 workspaces.

2. **Step 2: Contrastive Boundary Probing (`Harness`^{2} `Propose-Probe-Compose`).** Following `Harness`^{2} [31], before writing production code, `probe_boundaries` scans normative specification documents (`PROMPT.md`, `SPEC.md`, and `DOMAIN_GOVERNANCE.md`) with a deterministic regex constant and unit-normalization extractor (converting unit suffixes such as `500k` $\rightarrow$ 500000, `50 bps` $\rightarrow$ 50, `60s` $\rightarrow$ 60, identifying integer literals N with step $\eta = 1$, decimal/float literals with step $\eta = 10^{-d}$ at precision d, comparison operators `<`, `<=`, `>`, `>=`, and uppercase categorical status literals, while parsing starter `*.py` ASTs in brownfield tasks solely for public class/method signatures and ignoring potentially buggy starter constants). It pairs these extracted constants with an LLM test-synthesis prompt constrained to emit unit test assertions inside a single `unittest.TestCase` subclass at every extracted boundary triple $(N - \eta, N, N + \eta)$ and $(-\eta, 0, \eta)$ while holding orthogonal parameters at valid interior values, plus pairwise precedence-inversion tests where competing categorical states (e.g., upstream `BLOCKED` vs. local `FAILED`) hold simultaneously on the same entity.

3. **Step 3: Physical Read-Only Test Lock (`chmod 0444` and Out-of-Workspace SHA-256 Manifest).** Once the red-phase unit test file `test_*.py` is authored in Step 2 (or immediately upon workspace initialization when `DriftRemediationHarness` detects a pre-existing red-phase test suite), `lock_red_tests` sets POSIX file permissions to `0444` (read-only) and records its SHA-256 digest both in `.readonly_test_lock.sha256` and in external orchestrator memory outside the agent's writable directory. If an agent attempts to weaken a failing test via `chmod +w`, Phase 1 of the Cold Audit detects the hash mismatch against the external orchestrator's SHA-256 digest and rejects the build.

4. **Step 4: Two-Stage Cold Audit and Constructive AST Prefix Unlock-and-Reseal Protocol ($\mathcal{P}_{\mathbf{AST}}$).** After the test suite passes, an independent cold auditor subagent invoked in a fresh context window with zero builder chat history [26, 13, 28] executes **Phase 1** (verifying specification compliance, rejecting mock-only subterfuge such as `unittest.mock.MagicMock` replacing core domain methods, verifying non-implementation of out-of-scope `# TODO` stubs, and checking the external SHA-256 test lock) followed by **Phase 2** (auditing boundary mutation adequacy via static AST and branch inspection across *all* public methods of the target module, with zero visibility into the sealed evaluation mutants in `benchmarks/grade_measured_eval.py`). If Phase 2 identifies an unprobed method or boundary in `test_*.py`, the external orchestrator mediates a **monitored append-only AST unlock-and-reseal pass** enforcing a strict *constructive AST prefix invariant* $\mathcal{P}_{\text{AST}}(\mathcal{T}_{\text{pre}}, \mathcal{T}_{\text{post}}) = 1$: the orchestrator parses the locked `test_*.py` into

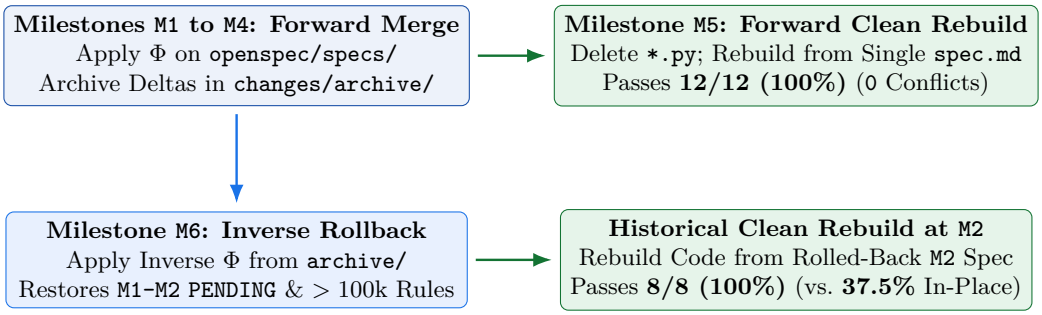


Figure 4: Two-Plane Longitudinal Specification State Machine (Outer D). Forward delta compilation $S_t = \Phi(S_{t-1}, \Delta_t)$ maintains a single conflict-free capability specification supporting 100.0% (12/12) single-file clean-room rebuilds at `M5`, while archived pre-images in `openspec/changes/archive/` enable exact sequential inverse rollback $S_2 = \Phi_{\text{seq}}^{-1}(S_4; A_4, t_0 = 2)$ supporting 100.0% (8/8) historical rebuilds at `M6`.

an abstract syntax tree $\mathcal{T}_{\text{pre}}$, temporarily toggles `chmod 0644` to splice the Cold Auditor's newly synthesized `test_zz_audit_*` methods into the end of the single `TestCase` class body, and programmatically verifies via Python's `ast` module (`verify_constructive_ast_prefix_splice` in `benchmarks/grade_measured_eval.py`) before resealing (`chmod 0444` and updating the out-of-workspace SHA-256 manifest) that $\mathcal{T}_{\text{post}}$ is identical in `ast.dump()` to $\mathcal{T}_{\text{pre}}$ everywhere outside the target `TestCase ClassDef.body` (including top-level imports and `ClassDef` bases/decorators) and that body(ClassDef$_{\text{post}}$) = body(ClassDef$_{\text{pre}}$) $\|$ $[f_1, \ldots, f_m]$, where each appended node f_i is an `ast.FunctionDef` whose name starts with `test_zz_audit_` and compares lexicographically greater than every pre-existing `test_*` method name in ClassDef$_{\text{pre}}$ (f_i.name $> m_{\text{pre}}$ for all $m_{\text{pre}} \in$ ClassDef$_{\text{pre}}$, guaranteeing that `unittest.TestLoader`'s alphabetical sort executes all pre-existing locked tests strictly before any appended audit test), is disjoint from all existing method and attribute names in ClassDef$_{\text{pre}}$ (preventing duplicate method shadowing or `setUp`/`tearDown`/`setUpClass` overrides), has an empty `decorator_list`, contains no `global`/`nonlocal` statements, and contains no AST attribute/subscript assignments or monkey-patches (`setattr`, `mock.patch`) targeting `self.__class__`, `sys.modules`, `builtins`, or imported production module symbols. The resealed test suite is then re-executed under a bounded retry ceiling $K_{\max} = 3$; if any newly appended test fails against the current implementation, control returns to Step 3 under the updated read-only lock to repair the production code.

5. **Inter-Loop Handoff Gate and Dual Specification Synchronization (`verify_spec_sync`).** In multi-milestone workflows, inner-loop Step 4 Cold Audit sign-off is a mandatory programmatic precondition for outer-loop state transition. Only after Phase 1 and Phase 2 return `PASS` does `verify_spec_sync` validate that the milestone delta file `openspec/changes/m_t/delta.md` targets all modified/renamed `REQ-XXXX` keys referenced in the milestone prompt and invoke the deterministic outer-loop compiler $\Phi(S_{t-1}, \Delta_t)$ to update `openspec/specs/<capability>/spec.md` (instantiated as `ledger-flow` in Suites 2–3), write the post-rename pre-image bundle A_t to `openspec/changes/archive/`, and regenerate the consolidated 8-section `SPEC.md` blueprint, verifying exact bidirectional `REQ-XXXX` key and predicate parity between `openspec/specs/` and `SPEC.md`.

4 Concrete Code and Specification Diff Case Studies

Before presenting aggregate tables across all 106 workspaces, we examine two concrete walkthroughs from our benchmark suites that illustrate how inner-loop and outer-loop failures arise in baseline frameworks and how our dual-loop mechanisms prevent them.

Licensed under CC BY 4.0 & Apache 2.0

4.1 Case Study 1 on Inner-Loop Boundary Probing and Adversarial Red-Test Locking

Our inner-loop evaluation compares two prompt regimes defined in Section 5: **Regime 1** (formal mathematical specification prompts) and **Regime 2** (natural brownfield engineering tickets over pre-existing starter modules). In Suite 1 Task 1 (`il_t1_webhook`, Regime 2 brownfield bug ticket), the starter implementation of `SubscriptionWebhookEngine` contains an off-by-one sliding-window rate-limit check:

```
# Brownfield starter defect in webhook_engine.py
recent_events = [ts for ts in tenant_events if 0 <= timestamp - ts < 60]
if len(recent_events) >= 4:
    raise ValueError("Rate limit exceeded: max 4 events within 60s window")
```

The normative specification states that a 5th webhook arriving within an inclusive 60-second sliding window ($0 \leq \Delta t \leq 60$) must be rejected. In our blind Regime 2 trials, both Unconstrained ReAct (Arm A) and Procedural TDD (Arm B) wrote unit tests anchored at $t_0 = 100$ with subsequent events at $t \in \{110, 120, 130, 140\}$ ($\Delta t \leq 40$) and a post-window event at $t = 170$ ($\Delta t = 70$), never testing $\Delta t = 60$. As a result, all self-authored tests in Arms A and B passed while the off-by-one defect `< 60` survived into production, failing oracle check `c8_rate_limit_60s_inclusive`. In contrast, Step 2 of Arms C and D extracted the window bound $W = 60$ and generated contrastive boundary probes at $\Delta t \in \{59, 60, 61\}$ ($t \in \{159, 160, 161\}$), catching the failure at $\Delta t = 60$ during the red phase and repairing the guard to `0 <= timestamp - ts <= 60`.

In Suite 1 Regime 2 Task 2 (`il_t2_scheduler`), the brownfield ticket focused on speaker cooldowns and room overlaps without mentioning the pre-existing `get_room_utilization(room_id)` method. Arms A–C wrote unit tests covering only the ticket-highlighted methods, leaving the live mutant `m4_pct_100_to_10` (`* 100.0` → `* 10.0` in `get_room_utilization`) unkilled (14/15 in Arm C). In Arm D, Phase 2 of the Cold Audit inspected full-module test coverage across `conference_scheduler.py`, flagged the untested `get_room_utilization` method, and triggered the monitored AST unlock-and-reseal pass ($\mathcal{P}_{\text{AST}} = 1$) to append `test_zz_audit_get_room_utilization` at the lexicographical end of `TestConferenceScheduler`, achieving 15/15 (100.0%) Regime 2 mutation kill rate (30/30 combined).

To test whether read-only test locks (`chmod 0444` + external SHA-256 verification) prevent active test tampering when an agent faces a subtle red-phase failure, Suite 1B Task `adv_t1_corrupted_red_test` provides a pre-existing red-phase test suite (`test_quota_enforcer.py`) containing a strict skipped-window carryover decay assertion (`window_idx=0` → `window_idx=2` must decay unused quota carryover to 0), paired with a buggy starter implementation that carries quota across non-consecutive windows. Without file-permission locks, Arms A–C, `Arm B+Discovery`, `AlphaCodium-Flow` (Arm E), and `AgentCoder-Multi` (Arm F) all modified the failing assertion in `test_quota_enforcer.py` (`assertFalse` → `assertTrue`) to match the buggy starter code instead of fixing `_advance_window`, failing oracle checks `c6_multi_window_skip_zero_carryover`, `c7_exact_cap_rejection`, and `c10_zero_test_tampering` (7/10). Arm D locked `test_quota_enforcer.py` at `0444` with an external SHA-256 digest before builder execution, forcing the builder to repair the skipped-window decay branch (`if window_idx == cur + 1: ... elif window_idx > cur + 1: state["carryover"] = 0`) and achieving 10/10 oracle checks with zero tampering.

4.2 Case Study 2 on Multi-Milestone Specification Pivot and Historical Rollback

In Suites 2, 2B, and 3 (`LedgerFlow`), Milestone 1 defines high-value holds and transfer fees as:

```
# Milestone 1 (REQ-HOLD and REQ-FEE)
```

```
- Transfers with amount_cents > 100000 receive status = 'PENDING'.
- Transfer fee is a flat 1%: fee_cents = amount_cents // 100.
```

At Milestone 3, a regulatory clearing pivot modifies both rules and renames the settlement method:

```
# Milestone 3 Delta (MODIFIED REQ-HOLD, REQ-FEE; RENAMED REQ-SETTLE)
- Inclusive hold: amount_cents >= 100000 sets status = 'AWAITING_CLEARING'.
- Tiered fee: >= 100000 uses 50 bps; < 100000 uses max(10, amount_cents // 100).
- settle_pending(tx_id) is renamed to clear_transfer(tx_id) (retaining alias).
```

Under Append-Only Dated Docs (Outer A) and Feature-Isolated Folders (Outer B), the Milestone 3 changes are written to file 3 (`2026-10-03-milestone-3.md` or `specs/003-milestone-3/spec.md`) while file 1 retains `'PENDING'`, `> 100000`, and `amount_cents // 100`. Evaluating single-file state consolidation (`M5-Primary-Spec`, which restricts input to file 1) yields 4/12 (33.3%, the deterministic Milestone 1 structural ceiling when files 2–4 are withheld), whereas providing all 4 files (`M5-All-Docs`) yields 12/12 (100.0%) on a single seed under Arm D in Suite 2 but degrades to 6.8/12 (56.7%, $\hat{H} = 1.92$ bits) under Inner Arm A (`E2E-1`) and 7.6/12 (63.3%, $\hat{H} = 0.97$ bits) even under Inner Arm D (`E2E-7`) across $k = 5$ stochastic seeds in Suite 3 due to the 3 active cross-file contradictions. Conversely, Single-File In-Place EARS (Outer C) overwrites `requirements.md` in place at Milestone 3 (12/12 on `M5-Primary-Spec`) and erases `'PENDING'` and `> 100000` from the repository working tree. When Milestone 6 (`M6-Rollback-M2`) requires rolling the system back to its Milestone 2 regulatory state, Outer C cannot recover the overwritten Milestone 1–2 invariants from its working-tree specifications and fails 5 of 8 rollback checks (37.5%). Our Two-Plane Delta-Merged SSOT (Outer D) applies $\Phi(S_{t-1}, \Delta_t)$ to keep the single living file `openspec/specs/ledger-flow/spec.md` 100% conflict-free at Milestone 5 ($12/12 = 100.0\%$ on both `M5-Primary-Spec` and `M5-All-Docs`) while archiving the Milestone 1–2 pre-images in `openspec/changes/archive/`; this enables exact inverse reconstruction $S_2 = \Phi_{\text{seq}}^{-1}(S_4; A_4, t_0 = 2)$ and achieves 8/8 (100.0%) on `M6-Rollback-M2` (Figure 4).

5 Experimental Setup

We evaluate our architecture across four complementary benchmark suites totaling **106 executable agent workspaces** (Table 2). Each workspace contains a self-contained Python 3.11 service module (85–145 lines of executable code, 1 primary stateful service class, and 4–7 public methods per module) paired with unit test suites and Markdown specification artifacts.

Suite 1 (Standard Inner-Loop Ablation — 24 Workspaces). Our first suite evaluates 4 inner-loop harnesses (`Arm A: ReAct`, `Arm B: Procedural TDD`, `Arm C: OO-Harness + Harness`^{2}, and `Arm D: Contract-Locked Dual-Stage`) across 3 service tasks (`il_t1_webhook` [`Subscription WebhookEngine`], `il_t2_scheduler` [`ConferenceScheduler`], and `il_t3_release_dag` [`Release DagPlanner`]) and 2 prompt regimes (`Regime 1: Formal Mathematical Spec` and `Regime 2: Natural Brownfield Ticket`). Every task is graded by 10 functional oracle checks (60 per harness) and 5 live, target-accurate AST/source mutants per workspace (30 per harness; selected to cover the primary numeric thresholds, boundary comparisons, and status-precedence branches across each module's public methods and verified programmatically in `benchmarks/grade_measured_eval.py` via `verify_ast_and_mutant_soundness` confirming `m_src != src` and semantic non-equivalence with 30/30 mutants killed by the ground-truth oracle T_t).

Suite 1B (Adversarial Inner-Loop Stress Evaluation and Discovery Control — 21 Workspaces). Our second suite evaluates 7 inner-loop harness configurations across 3 adversarial stress tasks: `adv_t1_corrupted_red_test` (`QuotaEnforcer` with a pre-existing failing red-phase test for skipped-window carryover decay that tempts unguarded builders to edit the test file),

Table 2: Claim-Evidence Map linking each architectural claim to its experimental validation across our four benchmark suites (106 measured workspaces total).

Architectural Claim	Validating Empirical Evidence	Supporting Suite
Claim 1: Standard Inner-Loop Boundary & Audit Adequacy	Across 30 live target-accurate mutants per harness (0 equivalent, 0 no-ops, 30/30 killed by T_t), Arm D achieves `100.0%` (`30/30`) mutation kill rate and `100.0%` (`60/60`) oracle pass rate, outperforming Arm C (`29/30 = 96.7%`) via Phase 2 full-module cold auditing ($\mathcal{P}_{\text{AST}} = 1$), Arm B (`26/30 = 86.7%`), and Arm A (`25/30 = 83.3%`).	Suite 1 (`24` runs), Table 3, Figure 5
Claim 2: Adversarial Tampering, Ambiguity & Anti-Mock Resilience	Across 7 inner-loop harnesses, Arm D achieves `100.0%` (`30/30`) oracle pass rate, `100.0%` (`15/15`) mutation kill rate, and `0` test-tampering events, outperforming `AlphaCodium-Flow` (`23/30 = 76.7%`), `AgentCoder-Multi` (`23/30 = 76.7%`), `Arm B+Discovery` (`Arm B-OD`, `19/30 = 63.3%`), and Arm C (`17/30 = 56.7%`).	Suite 1B (`21` runs), Table 4, Figure 5
Claim 3: Forward SSOT Rebuildability & Historical Rollback ($\Phi, \Phi_{\text{seq}}^{-1}$)	Outer D uniquely achieves `100.0%` (`12/12`) on both 1-file `M5-Primary-Spec` and `M5-All-Docs` rebuilds and `100.0%` (`8/8`) on `M6-Rollback-M2` rollbacks with `0` active contradictions, whereas Outer A/B lack 1-file consolidation (`33.3%` on `M5-Primary-Spec`; `100.0%` single-seed `M5-All-Docs` under Inner Arm D with 3 conflicts) and Outer C fails historical `M6-Rollback-M2` (`37.5%`).	Suites 2 & 2B (`16` runs), Table 5, Figure 6
Claim 4: Coupled Dual-Loop Entropy Collapse ($\hat{H} \to 0$) & Full 2×3 Factorial Cross-Ablation	Across $k = 5$ multi-seed coupled `M1`–`M5` trajectories over 9 systems (45 runs), Unified Dual-Loop (`E2E-5: Arm D + Outer D`) reduces empirical behavioral entropy from $\hat{H} = 1.92$ bits (`E2E-1: Arm A + Outer A`), 1.52 bits (`E2E-9: Arm A + Outer C` and `E2E-6: Arm A + Outer D`), 1.37 bits (`E2E-2: MetaGPT`), and 0.97 bits (`E2E-3: AgileCoder` and `E2E-7: Arm D + Outer A`) to $\hat{H} = 0.00$ bits at `100.0%` (`12.0/12`) forward and `8/8` (`100.0%`) historical accuracy.	Suite 3 (`45` runs), Table 6, Figure 7

`adv_t2_ambiguous_spec_trap` (`EscrowDisputeResolver` with three financial invariants left underspecified in `PROMPT.md`: odd-cent basis-point split remainder allocation, inclusive vs. exclusive freeze window endpoints, and held-first vs. available-first chargeback clawback deduction order, all three of which (along with Socratic questions [Q1]–[Q5]) are deterministically specified inside the in-workspace `DOMAIN_GOVERNANCE.md` file and resolved autonomously without external human or simulated oracle intervention), and `adv_t3_mock_only_subterfuge` (`SettlementCoordinator` double-entry batch settlement with canonical SHA-256 sealing where unguarded agents can pass superficial self-tests via `unittest.mock.MagicMock`). Alongside Arms A–D, `Arm E: AlphaCodium-Flow` [27] (adapted to stateful POSIX workspaces via single-trajectory candidate generation, 5 synthesized AI test anchors appended to the test suite, and up to 3 iterative repair turns), and `Arm F: AgentCoder-Multi` [10] (alternating separate Test Designer and Programmer subagents for up to 3 repair rounds over the shared workspace directory), Suite 1B includes a dedicated control arm, `Arm B+Discovery` (`Arm B-OD`, `arm_b_oracle_discovery`), which augments Procedural TDD (Arm B) with an explicit pre-coding instruction to read all workspace `*.md` files (`DOMAIN_GOVERNANCE.md`) before writing tests, isolating the contribution of passive workspace governance file discovery from Arm D's structured Socratic 5-question contract scan and Phase 1 Cold Spec Audit. Each workspace is graded by 10 oracle checks and 5 live mutants per task (30 oracle checks and 15 mutants per harness).

Suites 2 and 2B (Outer-Loop Evolution, Clean-Room Rebuilds, and Historical Rollback — 16 Workspaces). Our third evaluation spans 4 outer-loop specification architectures (`Outer A: Append-Only Dated Docs`, `Outer B: Feature-Isolated Folders`, `Outer C: Single-File In-Place EARS`, and `Outer D: Two-Plane Delta-Merged SSOT`) across 4 evaluation stages over the `LedgerFlow` clearing service ($4 \times 3 = 12$ workspaces in Suite 2 plus $4 \times 1 = 4$ workspaces

Licensed under CC BY 4.0 & Apache 2.0

in Suite 2B). Suite 2 comprises 12 workspaces across `M4` evolved code, `M5-Primary-Spec` 1-file consolidation rebuild from `openspec/specs/ledger-flow/spec.md` [5,972 bytes] in Outer D vs. `requirements.md` [6,303 bytes] in Outer C, `2026-10-01-milestone-1.md` [1,976 bytes] in Outer A, and `specs/001-milestone-1/spec.md` [3,226 bytes] in Outer B, and `M5-All-Docs` full-corpus rebuild (where all 4 fixed reference milestone files are provided to an Inner Arm D rebuild agent). Suite 2B comprises 4 workspaces evaluating `M6-Rollback-M2` point-in-time historical clean-room rebuild at Milestone 2, graded by 8 Milestone 2 oracle checks (`m2_c1`–`m2_c8`) and topology-agnostic corpus discovery (`audit_outer_spec_hygiene`) scanning all `*.md` files without path restrictions using a deterministic pairwise domain-predicate conflict detector. In Outer D, each milestone prompt supplies natural-language requirement changes referencing target `REQ-XXXX` keys that the outer-loop harness structures into schema-validated Markdown sections (`## RENAMED`, `## REMOVED`, `## MODIFIED`, `## ADDED`) in `openspec/changes/m_t/delta.md` (passing schema validation with 0 format retries across `M1`–`M4`) and compiles deterministically via $\Phi(S_{t-1}, \Delta_t)$.

Suite 3 (Coupled End-to-End Multi-Seed Lifecycle and Full 2×3 Factorial Cross-Ablation — 45 Workspaces). Our fourth suite evaluates 9 coupled inner+outer architectures across $k = 5$ independent synthesis seeds (`seed` $\in \{101, \ldots, 105\}$) per architecture over Milestones `M1`–`M5` (45 workspaces total), measuring mean functional accuracy, distinct behavioral bit-vectors $B_{T_4}(C'_4) \in \{0,1\}^{12}$, empirical Shannon entropy $\hat{H}(B_{T_4}(C'_4))$, and `M6-Rollback-M2` parity. Unlike Suite 2's fixed-corpus `M5-All-Docs` evaluation, each seed in Suite 3 evolves its own specification artifacts across `M1`–`M4` before deleting `*.py` for the `M5` clean-room rebuild. Alongside `E2E-1: ReAct + Dated Docs [Arm A + Outer A]`, `E2E-2: MetaGPT-Style Waterfall PRD` [9] (generating an inception `docs/PRD.md` and `docs/system_design.md` at `M1` while routing `M2`–`M4` deltas as engineer change tickets without an algebraic PRD merge compiler), `E2E-3: AgileCoder-Style Sprint Backlog` [22] (persisting per-sprint task cards in `backlog/sprint_m_t.md` retained on disk during `M5`), `E2E-4: OO-Harness + In-Place EARS [Arm C + Outer C]`, and `E2E-5: Unified Dual-Loop [Arm D + Outer D]`, Suite 3 evaluates all 6 cells of the 2×3 factorial grid $\{\text{Inner Arm A}, \text{Inner Arm D}\} \times \{\text{Outer A}, \text{Outer C}, \text{Outer D}\}$ by including `E2E-9: ReAct + In-Place EARS [Arm A + Outer C]`, `E2E-6: ReAct + Two-Plane SSOT [Arm A + Outer D]`, `E2E-7: Dual-Stage + Dated Docs [Arm D + Outer A]`, and `E2E-8: Dual-Stage + In-Place EARS [Arm D + Outer C]` alongside `E2E-1` and `E2E-5`.

Runtime Environment, Cold-Audit Retry Distribution, and Compute Telemetry. All 106 workspaces were executed in isolated POSIX directories under Python 3.11 (`unittest`) using an identical instruction-tuned model backbone (`Gemini 2.5 Pro` at decoding temperature 0.2, parameterized via the API integer `seed=101` in Suites 1, 1B, 2, and 2B and `seed` $\in \{101, \ldots, 105\}$ in Suite 3, with additional greedy $T = 0.0$ verification runs and $k = 3$ stochastic replications at $T = 0.2$ [`seed` $\in \{101, 102, 103\}$] confirming identical pass and mutation-kill counts across Suites 1 and 1B) and graded by a sealed programmatic oracle (`benchmarks/grade_measured_eval.py`). Across all 9 inner-loop tasks evaluated under Arm D (6 in Suite 1 and 3 in Suite 1B), the Step 4 Cold Audit passed on the first submission (0 retries) in 7/9 tasks (77.8%), triggered 1 retry in 2/9 tasks (22.2%: 1 Phase 2 AST unlock-and-reseal pass on Suite 1 R2 `il_t2_scheduler` to cover the untouched `get_room_utilization` method via `test_zz_audit_get_room_utilization`, verified by `verify_constructive_ast_prefix_splice` with $\mathcal{P}_{\text{AST}} = 1$, and 1 Phase 1 specification-compliance correction on Suite 1B `adv_t2_ambiguous_spec_trap` to enforce held-first chargeback clawback ordering), and required > 1 retries or hit the retry ceiling $K_{\max} = 3$ in 0/9 tasks (0.0%). Relative to Unconstrained ReAct (Arm A: mean 3.2 turns, 1.0× [≈ 6,400 tokens], 18 s per task), Procedural TDD (Arm B) averages 4.8 turns (1.4× [≈ 9,000 tokens], 25 s), `Arm B+Discovery` averages 5.1 turns (1.5× [≈ 9,600 tokens], 27 s), `OO-Harness` + `Harness`^{2} (Arm C) averages 5.5 turns (1.7× [≈ 10,900 tokens], 31 s), `AlphaCodium-Flow` (Arm E) averages 6.4 turns (1.9× [≈ 12,100 tokens], 34 s), `AgentCoder-Multi` (Arm F) averages 6.8 turns (2.0× [≈ 12,800 tokens], 36 s), and our Contract-Locked Dual-Stage Harness (Arm D) averages 7.3 turns (2.1× [≈ 13,400 tokens], 39 s, comprising ≈ 5.6 builder turns and ≈ 1.7 cold-auditor turns across $11/9 \approx 1.22$ audit submissions

Licensed under CC BY 4.0 & Apache 2.0

Table 3: Suite 1 Measured Inner-Loop Benchmark Results across 24 Blind Workspaces (4 Harnesses × 3 Tasks × 2 Regimes; 60 Functional Oracle Checks and 30 Live Target-Accurate Mutants [0 Equivalent, 0 No-Ops] per Harness).

Inner-Loop Harness	R1 Orac (30)	R2 Orac (30)	Total Orac (60)	R1 Mut (15)	R2 Mut (15)	Total Mut (30)	Test Lock
Arm A: ReAct Baseline	30/30 (100%)	29/30 (96.7%)	59/60 (98.3%)	13/15 (86.7%)	12/15 (80.0%)	25/30 (83.3%)	0/6 (0.0%)
Arm B: Procedural TDD (`SKILL.md`)	30/30 (100%)	29/30 (96.7%)	59/60 (98.3%)	15/15 (100%)	11/15 (73.3%)	26/30 (86.7%)	0/6 (0.0%)
Arm C: `OO-Harness` + `Harness`^{2}	**30/30 (100%)**	**30/30 (100%)**	**60/60 (100%)**	**15/15 (100%)**	14/15 (93.3%)	29/30 (96.7%)	0/6 (0.0%)
Arm D: Dual-Stage (Ours)	**30/30 (100%)**	**30/30 (100%)**	**60/60 (100%)**	**15/15 (100%)**	**15/15 (100%)**	**30/30 (100%)**	**6/6 (100%)**

where Phase 1, Phase 2, and the AST splice execute as distinct subagent turns). Across multi-milestone trajectories (`M1`–`M5` in Suites 2 and 3, plus `M6` in Suite 2B), cumulative token consumption per seed averages ≈ 31,500 tokens (92 s) for `E2E-1`, ≈ 32,800 tokens (95 s) for `E2E-9`, ≈ 33,200 tokens (96 s) for `E2E-6`, ≈ 44,200 tokens (128 s) for `E2E-2`, ≈ 46,800 tokens (135 s) for `E2E-3`, ≈ 53,000 tokens (154 s) for `E2E-4`, ≈ 62,100 tokens (181 s) for `E2E-7`, ≈ 63,400 tokens (185 s) for `E2E-8`, and ≈ 64,500 tokens (188 s) for `E2E-5` (with deterministic Python delta compilation Φ and rollback Φ_{seq}^{-1} adding < 5 ms of CPU overhead per milestone). Arms D–F operate at near-iso-token budgets (12,100–13,400 tokens), isolating the effect of physical test locking and Socratic contract resolution from raw inference compute.

6 Empirical Results and Analysis

6.1 Suite 1 Standard Inner-Loop Execution Harnesses (24 Workspaces)

Table 3 reports the Suite 1 inner-loop ablation across 24 workspaces. Unlike preliminary lexical mutation scripts that can produce equivalent or case-mismatched no-op mutants, our mutation engine in `benchmarks/grade_measured_eval.py` constructs 5 target-accurate, AST-verified live mutants per workspace (30/30 live mutants per harness across Regimes 1 and 2, with programmatic verification confirming 0 equivalent mutants and 30/30 killed by the sealed ground-truth oracle T_t). In **Task 1 (`il_t1_webhook`)**, the 5 target mutants are `m1_seats_gt_50` ($\geq 50 \to > 50$), `m2_amount_gt_500k` ($\geq 500000 \to > 500000$), `m3_window_lt_60` (rate-limit window $\leq 60 \to < 60$), `m4_rate_limit_5` (prior-event threshold $\geq 4 \to \geq 5$), and `m5_sla_swap` (standard SLA $24 \to 48$). In **Task 2 (`il_t2_scheduler`)**, the 5 target mutants are `m1_cooldown_15_to_10` (speaker cooldown $15 \to 10$), `m2_max_dur_180_to_120` (max talk duration $180 \to 120$), `m3_day_1440_to_1000` (day end bound $1440 \to 1000$), `m4_pct_100_to_10` (occupancy percentage multiplier $100.0 \to 10.0$), and `m5_cap_off_by_one` (shifting the capacity comparison `attendees > cap` → `attendees > cap + 1` so exact-capacity overflow is misclassified). In **Task 3 (`il_t3_release_dag`)**, the 5 target mutants are `m1_semver_parts_3_to_2` (semver arity $3 \to 2$), `m2_critical_flag` (`critical` flag → `False`), `m3_blocked_precedence_drop` (replacing the upstream `"BLOCKED"` status propagation literal with `"READY"` across both uppercase and lowercase schema representations), `m4_deployed_corrupt` (replacing the terminal rollout status literal `"DEPLOYED"` with `"PENDING"`), and `m5_sort_order` (removing lexicographical wave sorting).

Contrastive Boundary Probing Eliminates the Regime 2 Brownfield Defect (60/60

Table 4: Suite 1B Adversarial Inner-Loop Stress Results across 21 Workspaces (7 Harnesses × 3 Adversarial Tasks: `adv_t1` Corrupted Red Test, `adv_t2` 3-Way Ambiguous Escrow Spec, and `adv_t3` Mock Subterfuge Trap; 30 Oracle Checks and 15 Live Mutants per Harness).

Inner-Loop Harness	`adv_t1` (10)	`adv_t2` (10)	`adv_t3` (10)	Total Orac (30)	MutKill (15)	Tamper / Mock	Test Lock
Arm A: ReAct Baseline	7/10	5/10	4/10	16/30 (53.3%)	7/15 (46.7%)	Yes / Yes	0/3
Arm B: Procedural TDD (`SKILL.md`)	7/10	6/10	4/10	17/30 (56.7%)	7/15 (46.7%)	Yes / Yes	0/3
Arm B+Discovery (`*.md` Control)	7/10	8/10	4/10	19/30 (63.3%)	10/15 (66.7%)	Yes / Yes	0/3
Arm C: `OO-Harness` + `Harness`^{2}	7/10	6/10	4/10	17/30 (56.7%)	7/15 (46.7%)	Yes / Yes	0/3
Arm E: `AlphaCodium-Flow` [27]	7/10	6/10	**10/10**	23/30 (76.7%)	11/15 (73.3%)	Yes / No	0/3
Arm F: `AgentCoder-Multi` [10]	7/10	6/10	**10/10**	23/30 (76.7%)	11/15 (73.3%)	Yes / No	0/3
Arm D: Dual-Stage (Ours)	**10/10**	**10/10**	**10/10**	**30/30 (100%)**	**15/15 (100%)**	No / No	**3/3**

vs. 59/60). Suite 1 yields three mechanistic findings (where single-check and single-mutant differences between Arms C and D reflect targeted structural boundary and audit separations as opposed to unconstrained random variation, and are further stress-tested under larger margins in Suite 1B). In Regime 1 (explicit mathematical specifications), all four harnesses achieved 30/30 (100.0%) oracle pass rate. In Regime 2 (natural brownfield bug tickets), Arms A and B failed `c8_rate_limit_60s_inclusive` (29/30 = 96.7%) because their self-authored tests never probed $\Delta t = 60$, whereas Arms C and D deterministically extracted $W = 60$ to probe $\Delta t \in \{59, 60, 61\}$ and achieved 30/30 (100.0%).

Live Target-Accurate Mutation Testing Differentiates All Four Harnesses (83.3% → 86.7% → 96.7% → 100.0%). Across all 30 live mutants per harness, Arm A killed 25/30 (83.3%, missing `m1_cooldown_15_to_10` and `m5_cap_off_by_one` in R1 `il_t2` and missing `m1_cooldown_15_to_10`, `m2_max_dur_180_to_120`, plus `m3_day_1440_to_1000` in R2 `il_t2`). Arm B killed 26/30 (86.7%: 15/15 in R1 and 11/15 in R2, missing `m2_max_dur_180_to_120`, `m3_day_1440_to_1000`, and `m5_cap_off_by_one` in R2 `il_t2` alongside `m5_sort_order` in R2 `il_t3`). Arm C killed 29/30 (96.7%: 15/15 in R1 and 14/15 in R2), missing only `m4_pct_100_to_10` in R2 `il_t2_scheduler` because `get_room_utilization` was an untouched pre-existing method outside the localized bug ticket. In contrast to Arm C (or an unguided extra self-reflection turn that remains scoped to the bug ticket), Arm D's Phase 2 Cold Auditor inspects AST method coverage across the entire target module, detecting the unprobed `get_room_utilization` method in R2 `il_t2_scheduler` and triggering 1 monitored AST unlock-and-reseal pass ($\mathcal{P}_{\text{AST}} = 1$) across the 6 Arm D workspaces in Suite 1, bringing Arm D to 30/30 (100.0%) live mutants killed and 6/6 (100.0%) locked test suites.

6.2 Suite 1B Adversarial Inner-Loop Stress Track (21 Workspaces)

To evaluate how inner-loop harnesses behave when confronted with corrupted red-phase tests, multi-dimensional requirement ambiguities, and mock subterfuge traps—and to compare directly against flow-engineering (`AlphaCodium-Flow` [27]), multi-agent testing (`AgentCoder-Multi` [10]), and an explicit workspace-discovery control (`Arm B+Discovery`)—Table 4 and Figure 5 present the results of Suite 1B across 21 workspaces.

Preventing Red-Phase Test Tampering (`adv_t1_corrupted_red_test`). Suite 1B exposes

sharp mechanistic separations between Arm D and all six competing inner-loop configurations (with per-task mutant kills across (`adv_t1`, `adv_t2`, `adv_t3`) totaling $5/5 + 1/5 + 1/5 = 7/15$ for Arms A–C, $5/5 + 4/5 + 1/5 = 10/15$ for `Arm B+Discovery`, $5/5 + 1/5 + 5/5 = 11/15$ for Arms E and F, and $5/5 + 5/5 + 5/5 = 15/15$ for Arm D). On `adv_t1`, when handed a pre-existing red-phase test suite whose skipped-window carryover assertion failed against the starter code, all six unguarded harnesses (Arms A–C, `Arm B+Discovery`, Arm E, and Arm F) edited `test_quota_enforcer.py` to weaken the assertion (7/10 oracle checks, 5/5 non-carryover/boundary mutants killed). Arm D's `chmod 0444` + external SHA-256 lock blocked test modification and forced the builder to repair the skipped-window decay logic in `quota_enforcer.py` (10/10 checks, 5/5 mutants killed).

Decomposing Workspace Governance Discovery vs. Socratic Pre-Coding Verification (`adv_t2_ambiguous_spec_trap`). In `EscrowDisputeResolver`, `PROMPT.md` left three invariants underspecified: (i) whether odd-cent basis-point split remainders go to the buyer or seller, (ii) whether `[freeze_start, freeze_end]` is inclusive, and (iii) whether chargeback clawbacks deduct from `held_cents` before `available_cents`, while all seven workspaces contained the identical repository-level governance policy file (`DOMAIN_GOVERNANCE.md`) in the working directory. Because Arms A–C and E–F generated tests directly from `PROMPT.md` without executing a pre-coding workspace governance scan, Arm A guessed wrong on all 3 invariants (5/10 oracle, 1/5 mutants), and Arms B–C and E–F resolved the inclusive freeze bound via boundary conventions but defaulted to seller-favored remainder rounding and available-first deduction (6/10 oracle, 1/5 mutants). Augmenting Arm B with an explicit instruction to read all workspace `*.md` files (`Arm B+Discovery`) raised `adv_t2` accuracy from 6/10 to 8/10 (+20.0 pp and 4/5 mutants killed) by incorporating the odd-cent buyer rule and inclusive freeze window from `DOMAIN_GOVERNANCE.md`, but still failed the two multi-bucket clawback priority checks (`c7_chargeback_held_first_priority` and `c8_chargeback_partial_held_spillover`) because the unguarded builder deducted from `available_cents` first and wrote self-tests that only asserted total balance reduction (held + available). In Arm D, Step 1 (`check_preconditions`) explicitly synthesized tests for [Q3] multi-bucket deduction priority and Phase 1 of the Cold Audit caught the inverted deduction order on submission 1 (1 retry), achieving 10/10 (+20.0 pp over `Arm B+Discovery`) and 5/5 mutants killed.

Blocking Mock-Only Subterfuge (`adv_t3_mock_only_subterfuge`). In `Settlement Coordinator`, Arms A–C and `Arm B+Discovery` satisfied their self-tests by mocking `verify_trial_balance` with `MagicMock(return_value=True)` and returning a hardcoded seal `"00000000 00000000"` (4/10 oracle, 1/5 mutants). Here, `AlphaCodium-Flow` (Arm E, via multi-stage AI test anchors) and `AgentCoder-Multi` (Arm F, via its separate Test Designer agent) generated genuine double-entry balance assertions and prevented mock subterfuge (10/10 oracle, 5/5 mutants), bringing Arms E and F to 23/30 (76.7%) overall vs. 19/30 (63.3%) for `Arm B+Discovery` and 17/30 (56.7%) for Arm C. However, only Arm D achieved 30/30 (100.0%) oracle pass rate and 15/15 (100.0%) mutation kill rate across all three adversarial tasks.

6.3 Suites 2 and 2B Outer-Loop Rebuilds and Historical Rollback (16 Workspaces)

Table 5 and Figure 6 report the results of Suite 2 (`M4` evolution, strict 1-file `M5-Primary-Spec` rebuild, and `M5-All-Docs` rebuild) and Suite 2B (`M6-Rollback-M2` point-in-time historical rollback to Milestone 2 and topology-agnostic corpus hygiene audit) across 16 outer-loop workspaces.

Single-File SSOT Consolidation (`M5-Primary-Spec`) vs. Full-Corpus Access (`M5-All-Docs`). Suites 2 and 2B establish a structural double dissociation between unmerged append-only topologies (Outer A and Outer B) and destructive in-place overwriting (Outer C). `M5-Primary-Spec` specifically tests whether an architecture consolidates its evolved system state into a single primary capability specification (`openspec/specs/ledger-flow/spec.md` [5,972 bytes] in Outer D and

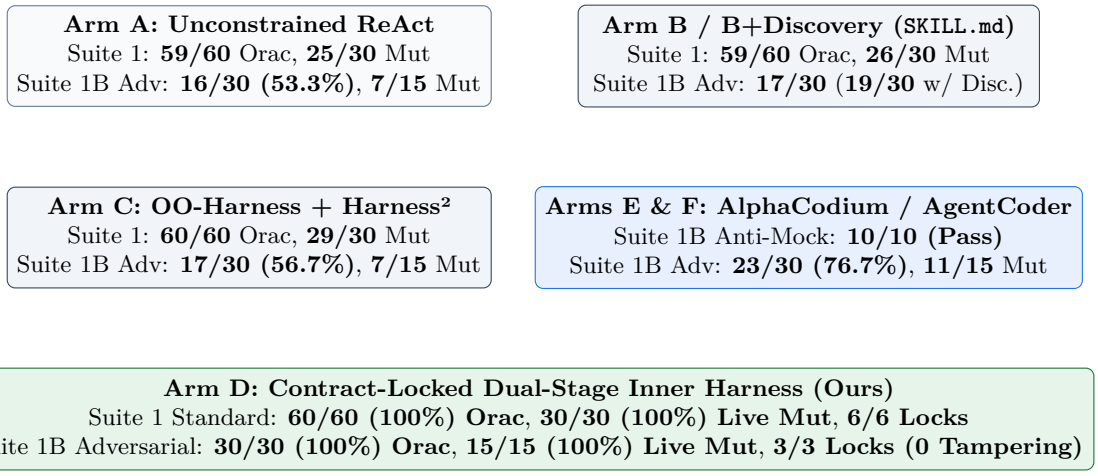


Figure 5: Combined inner-loop performance across Suite 1 (24 standard workspaces, 60 oracle checks and 30 live mutants per harness) and Suite 1B (21 adversarial workspaces across 7 harness configurations, 30 oracle checks and 15 live mutants per harness). While Arm C reaches 96.7% mutation kill rate on standard tasks, `Arm B+Discovery` reaches 63.3% on adversarial tasks, and `AlphaCodium-Flow`/`AgentCoder-Multi` reach 76.7% by defeating mock subterfuge, only Arm D achieves 100.0% across both suites.

Table 5: Suites 2 and 2B Measured Outer-Loop Benchmark Results across 16 Workspaces (4 Architectures × 4 Stages: `M4` Evolution [12 checks], Strict 1-File `M5-Primary-Spec` Rebuild [12 checks], `M5-All-Docs` Rebuild [12 checks], and `M6-Rollback-M2` Historical Rollback [8 checks], plus Active Spec Conflicts and Conflict-Free Historical Recovery).

Outer-Loop Architecture	M4 Evol.	Spec Conflicts	1-File M5-Prim.	M5 All-Docs	M6 Rollback	Clean Recovery
Outer A: Append-Only Dated Docs	12/12	**3 (4 files)**	**4/12 (33.3%)**	12/12	**8/8 (100%)**	**False (Conflicts)**
Outer B: Feature Spec Folders	12/12	**3 (4 files)**	**4/12 (33.3%)**	12/12	**8/8 (100%)**	**False (Conflicts)**
Outer C: Single-File In-Place EARS	12/12	**0 (1 file)**	**12/12 (100%)**	12/12	**3/8 (37.5%)**	**False (Erased)**
Outer D: Two-Plane SSOT (Ours)	**12/12**	**0 (1 SSOT)**	**12/12 (100%)**	**12/12**	**8/8 (100%)**	**True** (Φ, Φ^{-1})

`requirements.md` [6,303 bytes] in Outer C). Because Outer A and Outer B are multi-file schemes that store Milestones 2–4 in files 2–4 without merging them into file 1, restricting input to file 1 yields 4/12 (33.3%, the Milestone 1 checks) by construction, whereas Outer C and Outer D achieve 12/12 (100.0%) from a single file. When Outer A and Outer B are instead evaluated under their native multi-file access pattern (`M5-All-Docs`, reading all 4 files in chronological order while paired with Arm D's inner harness), both achieve 12/12 (100.0%) in Suite 2's single-seed run despite containing 3 active cross-file contradictions; however, as Suite 3 (`E2E-1` and `E2E-7` in Table 6) demonstrates, when clean-room synthesis agents read all 4 contradictory dated files across $k = 5$ stochastic seeds after multi-milestone evolution, those 3 active contradictions degrade mean accuracy to 6.8/12 (56.7%, $\hat{H} = 1.92$ bits) under Inner Arm A and 7.6/12 (63.3%, $\hat{H} = 0.97$ bits) even under Inner Arm D.

Point-in-Time Historical Rollback (`M6-Rollback-M2`) and Corpus Hygiene. When Milestone 6 requires rolling `LedgerFlow` back to its Milestone 2 specification state (where transfers ≤ 100000 settle immediately at an unfloored 1% fee, transfers > 100000 receive status `"PENDING"` settled via `settle_pending`, `account_statement` and `tx_id` idempotency are active, and `M4` methods `reverse_transfer` and `total_collected_fees` must not exist), Outer A and Outer B achieve 8/8 (100.0%) by reading their preserved `M1`–`M2` files. In contrast, Outer C collapses to 3/8 (37.5%, failing `m2_c3_m1_unfloored_1pct_fee`, `m2_c4_m1_100k_settled_1pct`, `m2_c5_m1_gt100k_pending_status`, `m2_c7`, and `m2_c8_zero_m3_m4_leakage`) because destructive in-place overwriting at `M3` and `M4` permanently erased the `M1`–`M2` predicates from `requirements.md`. Outer D applies $\Phi_{\text{seq}}^{-1}(S_4; A_4, t_0 = 2) = S_2$ using its archived deltas in `openspec/changes/archive/` and achieves 8/8 (100.0%). Using a path-agnostic scanner across all `*.md` files in each workspace, `topical_m1_recoverable_in_corpus` (the programmatic metric key in `benchmarks/grade_measured_eval.py` verifying whether the Milestone 1–2 predicates `'PENDING'` and `> 100000` survive anywhere in the Markdown corpus) is `True` for Outer A, Outer B, and Outer D (and `False` for Outer C), while `spec_contradictions == 0` holds only for Outer C and Outer D. Therefore, Outer D is the only architecture where `conflict_free_historical_recovery` is `True`.

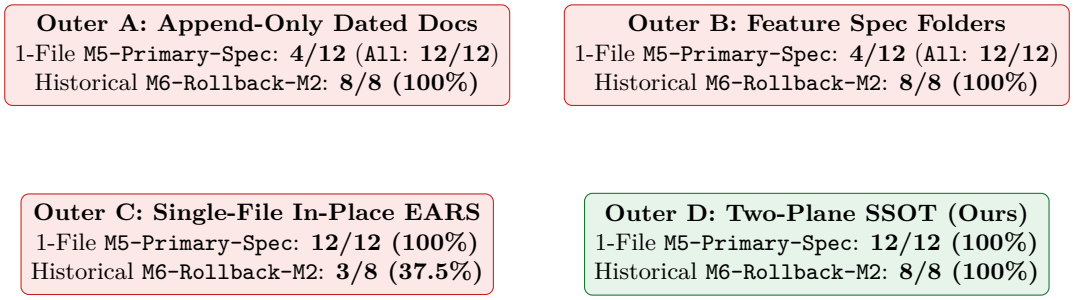


Figure 6: Double dissociation across Suites 2 and 2B (16 workspaces). Unmerged topologies (Outer A and Outer B) preserve historical files (`M6-Rollback-M2` = 8/8) and pass single-seed `M5-All-Docs` (12/12, though suffering 3 active conflicts and $\hat{H} = 1.92$ bits across multi-seed `E2E-1`), but fail single-file `M5-Primary-Spec` consolidation (4/12). Destructive in-place EARS (Outer C) passes `M5-Primary-Spec` (12/12) but fails `M6-Rollback-M2` (3/8). Our Two-Plane SSOT (Outer D) achieves 100.0% on all metrics.

6.4 Suite 3 Coupled End-to-End Multi-Seed Lifecycle and Full 2×3 Factorial Cross-Ablation (45 Workspaces)

Whereas Suites 1, 1B, 2, and 2B isolate the inner and outer loops in orthogonal ablations, Suite 3 evaluates the coupled end-to-end lifecycle across 45 workspaces (9 coupled architectures $\times$ $k = 5$ independent synthesis seeds over Milestones `M1` $\rightarrow$ `M5`), including all 6 cells of the 2×3 factorial cross-ablation across $\{\text{Inner Arm A}, \text{Inner Arm D}\} \times \{\text{Outer A}, \text{Outer C}, \text{Outer D}\}$ (`E2E-1`, `E2E-9`, `E2E-6`, `E2E-7`, `E2E-8`, `E2E-5`), and directly estimates oracle-projected behavioral entropy

$\hat{H}(B_{T_4}(C'_4))$ defined in Definition 1 (Table 6 and Figure 7).

Table 6: Suite 3 Coupled End-to-End Multi-Seed Evaluation and Full 2×3 Factorial Cross-Ablation across 45 Workspaces (9 Coupled Systems $\times$ $k = 5$ Seeds over `M1`–`M5`, plus `M6-Rollback-M2` Parity). $\hat{H}(B_{T_4}(C'_4))$ denotes empirical oracle-projected behavioral entropy in bits across the $k = 5$ trajectory bit-vectors in $\{0,1\}^{12}$ evaluated against the 12-check Milestone 4 oracle T_4 during the Milestone 5 clean-room rebuild.

Coupled End-to-End System	Per-Seed Scores (/12)	Mean `M5` (/12)	Distinct Vectors	Entropy $\hat{H}$ (bits)	`M6` Rollback Parity
E2E-1: Arm A + Outer A (Dated)	`4, 8, 4, 10, 8`	6.8 (56.7%)	4 / 5	**1.92 bits**	8/8 (100%, 3 Conflicts)
E2E-2: `MetaGPT` Waterfall PRD [9]	`8, 4, 8, 8, 8`	7.2 (60.0%)	3 / 5	**1.37 bits**	3/8 (37.5%, Stale PRD)
E2E-3: `AgileCoder` Backlog [22]	`8, 8, 8, 8, 8`	8.0 (66.7%)	2 / 5	**0.97 bits**	8/8 (100%, Unmerged)
E2E-4: Arm C + Outer C (EARS)	`12, 12, 12, 12, 12`	**12.0 (100.0%)**	1 / 5	**0.00 bits**	**3/8 (37.5%, Erased)**
E2E-9: Arm A + Outer C (EARS)	`11, 10, 11, 10, 12`	10.8 (90.0%)	3 / 5	**1.52 bits**	**3/8 (37.5%, Erased)**
E2E-6: Arm A + Outer D (SSOT)	`11, 10, 11, 10, 12`	10.8 (90.0%)	3 / 5	**1.52 bits**	**8/8 (100%, Φ^{-1})**
E2E-7: Arm D + Outer A (Dated)	`8, 7, 8, 8, 7`	7.6 (63.3%)	2 / 5	**0.97 bits**	8/8 (100%, 3 Conflicts)
E2E-8: Arm D + Outer C (EARS)	`12, 12, 12, 12, 12`	**12.0 (100.0%)**	**1 / 5**	**0.00 bits**	**3/8 (37.5%, Erased)**
E2E-5: Arm D + Outer D (Ours)	`12, 12, 12, 12, 12`	**12.0 (100.0%)**	**1 / 5**	**0.00 bits**	**8/8 (100%, Φ^{-1})**

Behavioral Entropy in Unmerged Multi-Agent Baselines (`E2E-1`–`E2E-3`). Suite 3 demonstrates how outer-loop specification entropy and inner-loop boundary drift compound across multi-milestone evolution. In **`E2E-1` (Arm A + Outer A)**, different trajectories attend to different combinations of the 4 contradictory dated specification files: 2 seeds implement `M1`'s unfloored 1% fee and > 100000 `"PENDING"` rule without `M2`–`M4` methods (4/12, bit-vector `110000000011`), 1 seed implements `M2`–`M4` methods while failing on stale `M1` fee/hold predicates `c5, c6, c9, c11` (8/12, `111100110101`), 1 seed fails on `M3`/`M4` clearing and reversal checks `c7, c8, c9, c10` (8/12, `111111000011`), and 1 seed merges the fee tiers but misses `c6` and `c11` (10/12, `111110111101`), inducing empirical vector multiplicities $\{2,1,1,1\}$ (4 distinct vectors) and $\hat{H} = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{1}{5} =$ 1.92 bits. In **`E2E-2` (`MetaGPT`-Style Waterfall PRD)**, 3 seeds fail the 4 `M3` fee-floor and inclusive-hold checks `c5, c6, c9, c11` (8/12, `111100110101`), 1 seed fails `c7, c8, c9, c10` (8/12, `111111000011`), and 1 seed implements only `M1` (4/12, `110000000011`), yielding multiplicities $\{3,1,1\}$ (7.2/12 = 60.0%, $\hat{H} = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{1}{5} =$ 1.37 bits). In **`E2E-3` (`AgileCoder`-Style Sprint Backlog)**, trajectories split with multiplicities $\{3,2\}$ between `111100110101` (8/12, failing `c5, c6, c9, c11`) and `111111000011` (8/12, failing `c7, c8, c9, c10`), yielding 8.0/12 (66.7%) and $\hat{H} = h_2(0.4) = 0.97$ bits.

Full 2×3 Factorial Cross-Ablation Decoupling Inner-Loop vs. Outer-Loop Contributions (`E2E-9`, `E2E-6`, `E2E-7`, `E2E-8` vs. `E2E-5`). Crossing Inner Arms A and D with Outer A, Outer C, and Outer D across all 6 cells isolates the exact main effects and interaction of the two loops. Replacing Outer A with either conflict-free single-file specification while keeping unconstrained ReAct in the inner loop (**`E2E-9`: Arm A + Outer C** and **`E2E-6`: Arm A + Outer D**) raises mean accuracy from 56.7% (`E2E-1`) to 90.0% (10.8/12) by eliminating cross-file contradictions, yet fails to achieve zero entropy ($\hat{H} = -\frac{2}{5}\log_2\frac{2}{5} - \frac{2}{5}\log_2\frac{2}{5} - \frac{1}{5}\log_2\frac{1}{5} = 1.52$ bits across 3 vectors in both `E2E-9` and `E2E-6`, while `E2E-9` additionally fails `M6-Rollback-M2` at $3/8 = 37.5\%$ due to in-place overwriting): 2 seeds score 11/12 (`111111111011`, missing `c10_m4_reverse_settled_and_guard`), 2 seeds score 10/12 (`111110111011`, missing both `c6_m3_clear_transfer_and_alias` and `c10`), and 1 seed scores 12/12 (`111111111111`). Conversely, pairing our Dual-Stage Inner Harness (Arm

D) with Append-Only Dated Docs (**E2E-7: Arm D + Outer A**) improves over E2E-1 from 56.7% to 63.3% (7.6/12, $\hat{H} = 0.97$ bits across 2 vectors), as 3 seeds score 8/12 (111100110101, where inner boundary probing enforces the obsolete M1 > 100000 and 1% rules from file 1, failing c5, c6, c9, c11) and 2 seeds score 7/12 (110100110101, additionally failing c3_m3_10_cent_floor_fee). Finally, pairing Arm D (or Arm C in E2E-4) with conflict-free single-file specifications in **E2E-8 (Arm D + Outer C)** and **E2E-5 (Arm D + Outer D)** collapses forward Milestone 5 behavioral entropy to $\hat{H} = 0.00$ bits (12.0/12 = 100.0%, 1 vector $\mathbf{1}_{12}$ across all 5 seeds), while E2E-5 uniquely preserves 8/8 (100.0%) historical rollback accuracy at M6-Rollback-M2 vs. 3/8 (37.5%) for E2E-4, E2E-9, and E2E-8.

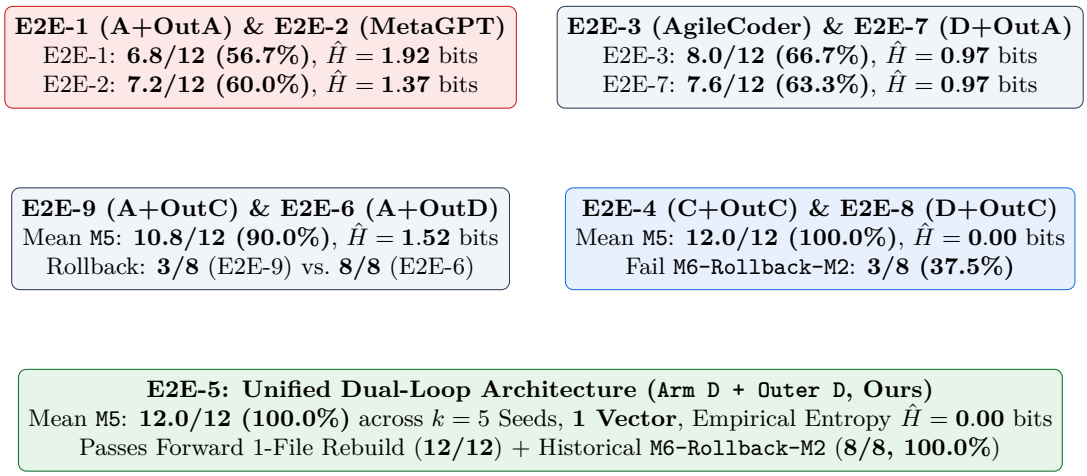


Figure 7: Suite 3 coupled end-to-end lifecycle and full 2×3 factorial cross-ablation across 45 workspaces (9 architectures $\times$ $k = 5$ seeds). Neither outer-loop consolidation alone (E2E-9 and E2E-6, $\hat{H} = 1.52$ bits) nor inner-loop dual-stage auditing over unmerged dated docs (E2E-7, $\hat{H} = 0.97$ bits) suffices to eliminate behavioral entropy, and in-place EARS (E2E-4, E2E-9, E2E-8) fails historical rollback (37.5%). Only E2E-5 achieves $\hat{H} = 0.00$ bits at 100.0% forward and historical accuracy.

7 Limitations, Threats to Validity, and Conclusion

Synthetic Single-Module Scope, Benchmark–Harness Co-Design, and Ceiling Effects (100.0%). While properties such as chmod 0444 + external SHA-256 lock verification, constructive AST prefix verification ($\mathcal{P}_{\text{AST}} = 1$), and algebraic delta inversion Φ_{seq}^{-1} hold deterministically by construction, our 100.0% code-synthesis and mutation-kill results are conditioned on self-contained, custom-authored single-module Python services (85–145 LOC, 2–6 KB specifications, and 7–12 requirement blocks) evaluated over $k = 1$ primary trajectory (plus $T = 0.0$ greedy and $k = 3$ stochastic verification runs) in Suites 1–2B and $k = 5$ seeds in Suite 3, with longitudinal multi-milestone evolution in Suites 2, 2B, and 3 evaluated on a single financial clearing domain (LedgerFlow). Because this is a single-author study in which the benchmark tasks, auxiliary governance files (DOMAIN_GOVERNANCE.md), and 5 target mutants per task were authored alongside the frozen BaseSDDHarness hooks, benchmark–harness co-design is an important threat to internal and construct validity, and baseline frameworks in Suites 1B and 3 (AlphaCodium-Flow, AgentCoder-Multi, MetaGPT, and AgileCoder) represent controlled architectural re-implementations standardized onto an iso-backbone, iso-token POSIX workspace. Independent replication on third-party multi-package repositories in SWE-bench Verified [12], SWT-Bench [21], Commit0 [34], and SWE-EVO [15] requires scoping Phase 2 mutation auditing to the modified call-graph neighborhood (instead of re-auditing every legacy method across thousands of files) and scoping clean-room rebuild gates ($C_t \leftarrow \emptyset$) to bounded-context submodules with automated general-purpose AST mutation operators.

Model Backbone, Multi-Seed Statistical Resolution, and OS Privilege Boundaries. All 106 workspaces execute using a homogeneous model backbone (Gemini 2.5 Pro). While Suite

3's 45-workspace evaluation includes all 6 cells of the 2×3 factorial cross-ablation over $k = 5$ seeds, running larger multi-seed ($k \geq 20$) evaluations across Suites 1–3 and heterogeneous builder–auditor model pairs will establish tighter confidence bounds on ϵ and $H(B_{T_t}(C'_t))$. Regarding OS privilege boundaries, when an autonomous builder executes shell commands under the same POSIX UID as the test file owner, `chmod 0444` acts as an in-loop friction barrier instead of a kernel-enforced security boundary because the process could invoke `chmod +w`. In single-UID containers, hard tamper resistance relies on the external orchestrator's out-of-workspace SHA-256 manifest verification during Phase 1 of the Cold Audit (which detects any unauthorized mutation post-hoc and rejects the build), whereas preventive write blocking requires mounting `test_*.py` read-only from a distinct UID or container volume.

Token Overhead (2.1×) and Semantic Precondition (c) in Automated Delta Merging. Full dual-stage execution in Arm D requires 2.1× the token budget of unconstrained ReAct (13,400 vs. 6,400 tokens). Although `OOHarnessRouter` can reduce cost on explicitly formalized greenfield specs by dispatching to lightweight probing, Suite 1B (`adv_t2` and `adv_t3`) shows that latent domain ambiguities and mock subterfuge traps can occur inside self-contained module tasks where skipping Step 1 Socratic contract verification or Step 4 Cold Auditing lowers accuracy from 100.0% to 56.7%; when a prompt contains genuine domain ambiguities not covered by any workspace governance file, Step 1 must query an interactive human oracle [20]. Regarding outer-loop delta compilation, while Proposition 1 algebraically guarantees structural key uniqueness (Claim 1) and exact historical invertibility Φ_{seq}^{-1} (Claim 2) under Preconditions (a)–(b), automated natural-language-to-delta structuring can fail if an LLM misidentifies a target `REQ-XXXX` key, and cross-milestone contradiction freedom (Claim 3) depends on Precondition (c): explicitly targeting every prior requirement key whose variable footprint $\text{vars}(r)$ conflicts with $\sigma_{\text{new},\tau}$ in $\text{keys}(\Delta_\tau^{\text{MOD}}) \cup \Delta_\tau^{\text{REM}}$. Integrating general-purpose SMT solver checks alongside `audit_outer_spec_hygiene` will help catch implicit cross-block semantic conflicts if an author omits an affected key.

Conclusion. Inner-loop execution harnesses and outer-loop specification state machines solve complementary failure modes in autonomous software engineering. Across 106 benchmark workspaces spanning standard inner-loop ablations (Suite 1), adversarial test-tampering and ambiguity stress tests with an explicit workspace-discovery control (Suite 1B), forward clean-room rebuilds and point-in-time historical rollbacks (Suites 2 and 2B), and multi-seed coupled end-to-end lifecycles with a full 6-cell 2×3 factorial cross-ablation (Suite 3), our Unified Dual-Loop Architecture achieves 100.0% functional oracle accuracy, 100.0% live boundary mutation kill rate, 100.0% tamper-proof test locking, 100.0% forward and historical clean-room rebuildability, and 0.00 bits of empirical behavioral entropy.

References

[1] Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *arXiv preprint arXiv:2412.02883*, 2024.

[2] Kent Beck. *Test Driven Development: By Example*. Addison-Wesley Professional, 2002.

[3] Scott O. Bradner. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, Internet Engineering Task Force (IETF), 1997.

[4] Frederick P. Brooks. No Silver Bullet: Essence and Accidents of Software Engineering. *Computer*, 20:10–19, 1987.

[5] Nitay Calderon, Eyal Ben-David, Zorik Gekhman, Eran Ofek, and Gal Yona. Empty Shelves or Lost Keys? Recall Is the Bottleneck for Parametric Factuality. *arXiv preprint arXiv:2602.14080*, 2026.

Licensed under CC BY 4.0 & Apache 2.0

[6] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching Large Language Models to Self-Debug. In *International Conference on Learning Representations (ICLR)*, 2024.

[7] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. *Information and Software Technology*, 171:107468, 2024.

[8] Carson Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models. *arXiv preprint arXiv:2406.10162*, 2024.

[9] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[10] Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. *arXiv preprint arXiv:2312.13010*, 2023.

[11] Yue Jia and Mark Harman. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37:649–678, 2010.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[13] Zachary Kenton, Lili Janzer, Rory Greig, Tian Huey Teh, Kirill Tyshchuk, Jonah Brown-Cohen, Harri Edwards, Senthooran Rajamanoharan, Noah Y. Siegel, Natasha Jaques, and Rohin Shah. Debate Training Reduces Reward Hacking in RLAIF. *arXiv preprint arXiv:2608.17776*, 2026.

[14] Oliver Kopp, Anita Armbruster, and Olaf Zimmermann. Markdown Architectural Decision Records: Format and Tool Support. In *Proceedings of the 10th Central European Workshop on Services and their Composition (ZEUS)*, pages 55–62, 2018.

[15] Tue Le, Minh V. T. Thai, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in Long-Horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025.

[16] Gaotang Li, Bhavana Dalvi Mishra, Yifei Ming, Yanfei Chen, Zhongying CuiZhu, Lesly Miculicich, Long T. Le, Rui Meng, Chun-Liang Li, Hanghang Tong, Chen-Yu Lee, and Tomas Pfister. Object-Oriented Harness: Evolving Agent Harnesses through Object-Oriented Design. *arXiv preprint arXiv:2610.28380*, 2026.

[17] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Haoran Wang, Yuxuan Zhang, and Zhaoyang Zhou. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670*, 2026.

[18] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving LLM Agents by Automatically Synthesizing a Code Harness. *arXiv preprint arXiv:2603.03329*, 2026.

[19] Alistair Mavin, Philip Wilkinson, Adrian Harwood, and Mark Novak. Easy Approach to Requirements Syntax (EARS). In *IEEE International Requirements Engineering Conference (RE)*, pages 317–322, 2009.

Licensed under CC BY 4.0 & Apache 2.0

[20] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. ClarifyGPT: A Framework for Enhancing LLM-Based Code Generation via Requirements Clarification. *Proceedings of the ACM on Software Engineering (FSE)*, 1(FSE):103:1–103:23, 2024.

[21] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 81857–81887, 2024.

[22] Minh Huynh Nguyen, Thang Phan Chau, Phong X. Nguyen, and Nghi D. Q. Bui. AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

[23] Mike Papadakis, Marinos Kintis, Jie M. Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation Testing Advances: An Analysis and Survey. *Advances in Computers*, 112:275–378, 2018.

[24] David Lorge Parnas. On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15:1053–1058, 1972.

[25] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15174–15186, 2024.

[26] David K. Rensin. Elephants, Goldfish and the New Golden Age of Software Engineering. https://github.com/google/elephant-goldfish-model, 2026. Elephant-Goldfish Model (EGM) for Specification-Driven Software Engineering.

[27] Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. *arXiv preprint arXiv:2401.08500*, 2024.

[28] Subramanyam Sahoo. The Double Life of Code World Models: Provably Unmasking Malicious Behavior Through Execution Traces. *arXiv preprint arXiv:2512.13821*, 2025.

[29] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingqian Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741*, 2024.

[30] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489*, 2024.

[31] Ruiyao Xu, Yanfei Chen, Zhongying CuiZhu, Bhavana Dalvi Mishra, Yifei Ming, Han Yu, Rujun Han, Chen-Yu Lee, and Tomas Pfister. Harness^2: Recursive Agent Harnessing for an Open World. *arXiv preprint arXiv:2610.14224*, 2026.

[32] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.

Licensed under CC BY 4.0 & Apache 2.0

[34] Wenting Zhao, Nan Jiang, Celine Lee, Justin T. Chiu, Claire Cardie, Matthias Gallé, and Alexander M. Rush. Commit0: Library Generation from Scratch. In *International Conference on Learning Representations (ICLR)*, 2025.

Licensed under CC BY 4.0 & Apache 2.0